

Domande LPP 3CFU

CT0050 First categories

Una categoria è formata essenzialmente da tre componenti:

- **Oggetti:** Le strutture matematiche in esame (es. insiemi, spazi vettoriali, gruppi o spazi topologici).
- **Morfismi (o Freccie):** Le relazioni o trasformazioni che collegano un oggetto all'altro (es. funzioni continue, mappe lineari).
- **Composizione:** La possibilità di unire due frecce consecutive $A \rightarrow B$ e $B \rightarrow C$ per ottenere una freccia diretta $A \rightarrow C$.

Ex1:

Prove that any pre-order is a category.

Hint 1.

- recall def. of pre-order
- show that reflexivity implies the existence of identity using transitivity
- show that transitivity is arrow composition

Soluzione:

Un pre-ordine $(C; \leq)$ è una categoria se soddisfa le definizioni di una categoria, con insieme C definito su una relazione $\leq$, ed è riflessivo e transitivo:

- Oggetti: un insieme di elementi oggetti della categoria (Gli oggetti non sono elementi di un insieme. Sono nodi astratti.)
- Morfismi (o frecce): esiste un unico morfismo (freccia) da un oggetto x a un oggetto y , denotato come $x \rightarrow y$, se e solo se $x \leq y$. Se $x \not\leq y$, non ci sono morfismi.
- Morfismo identità: la proprietà di riflessività di una relazione di preordine ha che per ogni elemento $x \in C$, si ha $x \leq x$. Questo implica l'esistenza di un morfismo speciale detto identità da x a se stesso, $\text{id}_x: x \rightarrow x$)
- $\forall a. a \leq a$
- quindi con la transitività si scopre che $A \rightarrow B$ ($A \leq B$) e $B \rightarrow A$ ($B \leq A$), allora si ha che per transitività $A \rightarrow A$ ($A \leq A$)
- Composizione dei morfismi: la transitività di un preordine definisce la composizione di morfismi: Dati 2 morfismi $f: x \rightarrow y$ e $g: y \rightarrow z$, esiste la composizione delle frecce: $g \circ f: x \rightarrow z$; in cui per transitività $\forall x, y, z. x \leq y \wedge y \leq z \Rightarrow x \leq z$; tale per cui soddisfa la proprietà:
- associatività: deriva dalla natura della relazione di preordine; per ogni $f \in x \rightarrow y$, $g \in y \rightarrow z$, $h \in z \rightarrow w$ la composizione: $h \circ (g \circ f) = (h \circ g) \circ f$ sono entrambe la freccia unica da x a w (poiché $x \leq y \leq z \leq w \Rightarrow x \leq w$)

Ex2:

Write an example of pre-order and the corresponding commuting diagram to show that the pre-order is a category.

Soluzione:

L'insieme dei numeri naturali $\mathbb{N}$ con la relazione " $\leq$ " è un pre-ordine, in cui si rispettano le proprietà della riflessività ($n \leq n$ per ogni n) e della transitività ($n \leq m$ e $m \leq p$ allora $n \leq p$)

Il diagramma commutativo si può fare con il classico esempio:

$A = \{0, 1, 2\}$ in cui il diagramma dimostra il pre-order $\{(0,0)(1,1)(2,2)(0,1)(1,2)(0,2)\}$ ma con le frecce

Ex3:

Prove that any partial-order is a category.

Soluzione:

Ogni partial-order è un pre-ordine con antisimmetria. Quindi vale quello scritto nel ex1 ma con l'aggiunta dell'assenza di cicli non banali; quindi $\forall a, b. a \leq b \wedge b \leq a \Rightarrow a = b$

Ex4:

Write an example of partial-order and the corresponding commuting diagram to show that the pre-order is a category. Highlight a possible graph that cannot be a sub-graph of the commuting diagram corresponding to the given example.

Soluzione:

Sia dato un insieme del tipo $A = \{a, b, c\}$ in cui $a \leq b$ e $b \leq c$, con b e c incomparabili. Un possibile grafo sarebbe con $a \rightarrow b$, $b \rightarrow c$ in cui valgono tutte le proprietà, però un grafo del tipo di prima ma con l'aggiunta $b \rightarrow c$ e $c \rightarrow b$ violerebbe l'antisimmetria

Ex5:

Prove that any total-order is a category.

Soluzione:

Ogni total-order è un partial-ordine in cui tra ogni coppia di oggetti c'è esattamente una freccia. Quindi vale quello scritto nel ex1 e ex3 ma con l'aggiunta per cui $\forall a, b. a \leq b \vee b \leq a$ (quindi $a \rightarrow b$ oppure $b \rightarrow a$)

Ex6:

Write an example of total-order and the corresponding commuting diagram to show that the pre-order is a category. Highlight a possible graph that cannot be a sub-graph of the commuting diagram corresponding to the given example.

Soluzione:

Sia dato un insieme del tipo $A = \{a, b, c\}$ in cui $a \leq b \leq c$. Un possibile grafo sarebbe con il seguente diagramma a frecce (in linea) $a \rightarrow b$, $b \rightarrow c$, $a \rightarrow c$ in cui valgono tutte le proprietà, però un grafo del tipo di prima ma emettendo uno di quelli non sarebbe valido (non un total-order)

Ex7:

Prove that at most one arrow exists between two objects of the commuting diagram corresponding to a pre-order.

Soluzione:

Se esistessero due frecce distinte $a \rightarrow b$, allora rappresenterebbero la stessa relazione $a \leq b$ (di pre-ordine); La definizione di pre-ordine dice che la relazione tra due elementi distinti "a" e "b" è unica (o esiste o non esiste)

In cui dato il diagramma di commutazione con le frecce $a \rightarrow b$, $a \rightarrow a$, $b \rightarrow b$ e $a \rightarrow b$; si avrebbe un non set (un insieme che contiene i duplicati, che è errato) $\{(a, a), (b, b), (a, b), (a, b)\}$

Ex8:

Prove that at most one arrow exists between two objects of the commuting diagram corresponding to a partial-order.

Soluzione:

Se esistessero due frecce $a \rightarrow b$ e $b \rightarrow a$ con $a \neq b$, allora si violerebbe l'antisimmetria e si avrebbe al più una freccia.

Inoltre come nel ex7 si ha che dato il diagramma di commutazione con le frecce $a \rightarrow b$, $a \rightarrow a$, $b \rightarrow b$ e $a \rightarrow b$; si avrebbe un non set (un insieme che contiene i duplicati, che è errato) $\{(a, a), (b, b), (a, b), (a, b)\}$

Ex9:

Prove that exactly one arrow exists between two objects of the commuting diagram corresponding to a total-order.

Soluzione:

La condizione di total-order garantisce che dati due oggetti distinti si ha che $a \leq b$ oppure $b \leq a$ (tutti gli elementi siano confrontabili), ma non entrambi a causa dell'antisimmetria del partial-order, pertanto esisterà una sola freccia tra i due oggetti

Inoltre dato il diagramma di commutazione con le frecce $a \rightarrow b$, $a \rightarrow a$, $b \rightarrow b$, $c \rightarrow c$ e $a \rightarrow c$; si avrebbe un non total-order (dov'è l'elemento c non è direttamente comparabile ad a o b, quindi no coppia di relazione) $\{(a, a), (b, b), (a, b), (a, c)\}$

Ex10:

Justify why we cannot say that every oriented graph is a category.

Soluzione:

Possono mancare identità o frecce di composizione

Esempio di grafo che non è una categoria: $a \rightarrow a$, $c \rightarrow c$, $d \rightarrow d$, $a \rightarrow b$, $a \rightarrow c$, $c \rightarrow d$; in cui manca un'identità freccia e una unique freccia che rappresenti la composizione

CT0100 Categorical monoids

Un monoide è una struttura algebrica definita come una tripla (es. $(X, \circ, e)$) con insieme X (di elementi), operazione associativa $\wedge$ (operazione sugli elementi, che definiscono la composizione) ed elemento neutro e (che definisce il morfismo identità)

Un monoide categoriale sarebbe un monoide trasformato in categoria con un solo oggetto $(\bullet)$, ogni elemento diventa una freccia (morfismo sullo stesso oggetto $(\bullet)$) (es. $x \in \mathbb{N} \mapsto (x): \bullet \rightarrow \bullet$; in cui $\bullet \xrightarrow{0} \bullet$, $\bullet \xrightarrow{1} \bullet$, ...)

Un monoide descrive azioni (morfismi) che si compongono (composizione) su una stessa cosa (oggetto)

Ex11:

Let $B = \{T, F\}$. Write the Categorical monoid (C) of the monoid $(B, \wedge, T)$

Soluzione:

Proprietà della categoria, quindi trasformazione da monoide a monoide categoriale

Oggetto: $\text{obj}(C) = \{\bullet\}$ (un solo oggetto)

Morfismi: $\text{hom}(C) = \{(T\wedge), (F\wedge) \mid (b\wedge): \bullet \rightarrow \bullet\}$; tale per cui per ogni $b \in B$

Composizione: $(b\wedge) \circ (b'\wedge) = (b'\wedge b)\wedge; \bullet \rightarrow (b\wedge) \bullet \rightarrow (b'\wedge) \bullet \implies \bullet \rightarrow (b'\wedge b)\wedge \bullet$

Identità: $(T\wedge)$ è l'identità;

$\bullet \rightarrow (b\wedge) \bullet \rightarrow (T\wedge) \bullet \implies \bullet \rightarrow (b\wedge) \bullet$

$\bullet \rightarrow (T\wedge) \bullet \rightarrow (b\wedge) \bullet \implies \bullet \rightarrow (b\wedge) \bullet$

Ex12:

Let $B = \{T, F\}$. Write the Categorical monoid (C) of the monoid $(B, \vee, F)$

Soluzione:

Proprietà della categoria, quindi trasformazione da monoide a monoide categoriale

Oggetto: $\text{obj}(C) = \{\bullet\}$ (un solo oggetto)

Morfismi: $\text{hom}(C) = \{(T\vee), (F\vee) \mid (b\vee): \bullet \rightarrow \bullet\}$; tale per cui per ogni $b \in B$

Composizione: $(b\vee) \circ (b'\vee) = (b'\vee b)\vee; \bullet \rightarrow (b\vee) \bullet \rightarrow (b'\vee) \bullet \implies \bullet \rightarrow (b'\vee b)\vee \bullet$

Identità: $(F\vee)$ è l'identità;

- $\bullet \rightarrow (bv) \bullet \rightarrow (Fv) \bullet == \bullet \rightarrow (bv) \bullet$
- $\bullet \rightarrow (Fv) \bullet \rightarrow (bv) \bullet == \bullet \rightarrow (bv) \bullet$

Ex13:

Let $B = \{T, F\}$. Write the Categorical monoid (C) of the monoid $(B, \oplus, F)$ ("exclusive or")

Soluzione:

Proprietà della categoria, quindi trasformazione da monoide a monoide categoriale

Oggetto: $\text{obj}(C) = \{\bullet\}$ (un solo oggetto)

Morfismi: $\text{hom}(C) = \{(T \oplus), (F \oplus) \mid (b \oplus): \bullet \rightarrow \bullet\}$; tale per cui per ogni $b \in B$

Composizione: $(b \oplus) \circ (b' \oplus) = (b' \oplus b) \oplus; \bullet \rightarrow (b \oplus) \bullet \rightarrow (b' \oplus) \bullet == \bullet \rightarrow (b' \oplus b) \oplus \bullet$

Identità: (Fv) è l'identità;

- $\bullet \rightarrow (b \oplus) \bullet \rightarrow (F \oplus) \bullet == \bullet \rightarrow (b \oplus) \bullet$
- $\bullet \rightarrow (F \oplus) \bullet \rightarrow (b \oplus) \bullet == \bullet \rightarrow (b \oplus) \bullet$

Ex14:

Write the Categorical monoid (C) of the monoid $(\Sigma^*, \cdot, \epsilon)$ (strings)

Soluzione:

Proprietà della categoria, quindi trasformazione da monoide a monoide categoriale

dove Σ rappresenta l'insieme delle lunghezze finite di stringhe (concatenando zero o più simboli di un alfabeto), Dato un esempio di un alphabet $\Sigma = \{a, b\}$, $\Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$

Oggetto: $\text{obj}(C) = \{\bullet\}$ (un solo oggetto)

Morfismi: $\text{hom}(C) = \{(a \cdot), (b \cdot), \dots \mid (\text{str} \cdot): \bullet \rightarrow \bullet\}$; tale per cui per ogni $\text{str} \in \Sigma^*$

Composizione (concatenazione): $(\text{str} \cdot) \circ (\text{str}' \cdot) = (\text{str}' \cdot \text{str}) \cdot; \bullet \rightarrow (\text{str} \cdot) \bullet \rightarrow (\text{str}' \cdot) \bullet == \bullet \rightarrow (\text{str}' \cdot \text{str}) \cdot$

Identità (stringa vuota): $(\epsilon \cdot)$ è l'identità;

- $\bullet \rightarrow (\text{str} \cdot) \bullet \rightarrow (\epsilon \cdot) \bullet == \bullet \rightarrow (\text{str} \cdot) \bullet$
- $\bullet \rightarrow (\epsilon \cdot) \bullet \rightarrow (\text{str} \cdot) \bullet == \bullet \rightarrow (\text{str} \cdot) \bullet$

Ex15:

Write the Categorical monoid (C) of the monoid $(\{ [a_1, \dots, a_n] \mid n \in \mathbb{N}\}, ++, [])$ (lists)

Soluzione:

Proprietà della categoria, quindi trasformazione da monoide a monoide categoriale

Definisco la "lista" come $\{ [a_1, \dots, a_n] \mid n \in \mathbb{N}\}$

Oggetto: $\text{obj}(C) = \{\bullet\}$ (un solo oggetto)

Morfismi: $\text{hom}(C) = \{([a_1] ++), ([a_2] ++), \dots \mid (l ++): \bullet \rightarrow \bullet\}$; tale per cui per ogni $l \in \text{lista}$

Composizione (concatenazione): $(l ++ \cdot) \circ (l' ++ \cdot) = (l' ++ l) ++ \cdot; \bullet \rightarrow (l ++ \cdot) \bullet \rightarrow (l' ++ \cdot) \bullet == \bullet \rightarrow (l' ++ l) ++ \cdot$

Identità (lista vuota): $([] ++)$ è l'identità;

$\bullet \rightarrow (l++) \bullet \rightarrow ([\]++) \bullet == \bullet \rightarrow (l++) \bullet$
 $\bullet \rightarrow ([\]++) \bullet \rightarrow (l++) \bullet == \bullet \rightarrow (l++) \bullet$

CT0150 Category Set and “beyond”

La categoria Set (per gli insiemi) ha le stesse definizioni e proprietà della categoria:

oggetti: tutti gli insiemi

morfismi: tutte le funzioni tra insiemi

identità: per ogni insieme X si ha la funzione identità $\text{id}_X(x) = x$

composizione: funzioni composte $(g \circ f)(x) = g(f(x))$

Si reinterpreta le definizioni di iniettività e suriettività per le categorie

Il prodotto cartesiano in Set è l'insieme (pattern) di $A \times B$ e con le sue proiezioni ($\pi_1: A \times B \rightarrow A$, $\pi_2: A \times B \rightarrow B$) (che sono morfismi) da una tripla ($A \times B$, $\pi_1: A \times B \rightarrow A$, $\pi_2: A \times B \rightarrow B$)

Una tripla dei candidati ha la seguente forma (Ca , f , g) dove Ca è il candidato (cioè un qualsiasi oggetto che pretende di “mettere insieme” A e B ed è dotato di due frecce verso di loro) mentre f , g sono le sue proiezioni che permettono di collegare Ca con A e B

Si introducono i candidati perché:

Perché la teoria delle categorie non può partire da $A \times B$ (livello astratto).

Motivo fondamentale: In una categoria astratta:

- non esistono “coppie”,
- non esiste una costruzione interna come $A \times B$,
- esistono solo oggetti e frecce.

Quindi non puoi dire: “il prodotto è $A \times B$ ”

Devi dire: “il prodotto è l'oggetto che batte tutti gli altri candidati”.

Il candidato Ca è un prodotto solo se esiste un'unica funzione $m: Ca \rightarrow A \times B$ (in categoria (Ca , $A \times B$)) che ricostruisce le proiezioni, cioè tale che $f = \pi_1 \circ m$ e $g = \pi_2 \circ m$ (fattorizza tutti gli altri candidati)

Ex16:

Recall the definition of monomorphism and how it is obtained as a generalization of the notion: “injective function” between two sets

Soluzione:

Un monomorfismo è un morfismo ($f: X \rightarrow Y$) tale per cui per ogni coppia di morfismi g, h si ha $f \circ g = f \circ h \Rightarrow g = h$ (quindi f è left-cancellable, cioè cancello la “freccia” sinistra dalla composizione)

Nella categoria Set (degli insiemi) la definizione generalizza la nozione di funzione iniettiva (quindi il monomorfismo è una funzione iniettiva categoriale), in cui i morfismi sono le funzioni e la condizione del monomorfismo è equivalente alla condizione della funzione iniettiva ($\forall x, w \in X. f(x) = f(w) \Rightarrow x = w$).

Per negazione: f non è iniettiva se esistono $x \neq w$ con $f(x)=f(w)$
quindi categorialmente:

- prendi un insieme Z con un solo elemento
- definisci due funzioni diverse; $g(z) = x$ e $h(z) = w$
- allora $f \circ g = f \circ h$ ma $g \neq h$ (non è left-cancellable quindi non è un monomorfismo)

Ex17:

Recall the definition of epimorphism and how it is obtained as a generalization of the notion: “surjective function” between two sets.

Soluzione:

Un epimorfismo è un morfismo ($f: X \rightarrow Y$) tale per cui per ogni coppia di morfismi g, h si ha $g \circ f = h \circ f \Rightarrow g = h$ (quindi f è right-cancellable, cioè cancello la “freccia” destra dalla composizione)

Nella categoria Set (degli insiemi) la definizione generalizza la nozione di funzione suriettiva (quindi il epimorfismo è una funzione suriettiva categoriale), in cui i morfismi sono le funzioni e la condizione del epimorfismo è equivalente alla condizione della funzione suriettiva ($\forall y \in Y. \exists x \in X. y = f(x)$).

Per negazione: f non è suriettiva se esiste $y \in Y$ che non è immagine di nessun x
quindi categorialmente:

- scegli due funzioni diverse $g, h: Y \rightarrow Z$
- le si fa coincidere su $\text{Im}(f)$ e le si fa diverse solo su $y \notin \text{Im}(f)$
- allora $g \circ f = h \circ f$ ma $g \neq h$ (non è right-cancellable quindi non è un epimorfismo)

Ex18:

Recall the definition of “Terminal object in a generic category” and how it is obtained as a generalization of the notion: “singleton set”.

Soluzione:

Un oggetto T è terminale se $\forall X, \exists ! X \rightarrow T$ (per ogni oggetto X esiste un unica freccia $X \rightarrow T$ (morfismo)); sono unici a meno di isomorfismo

Proof:

- T è terminale per T' e viceversa (morfismo $T \rightarrow T'$)
- T, T' hanno la loro identità
- la composizione delle frecce uniche $T \rightarrow T', T' \rightarrow T$ dev'essere id_T e viceversa dall'altra parte con $\text{id}_{T'}$
- $T \xrightarrow{f} T', T' \xrightarrow{g} T, T \xrightarrow{\text{id}_T} T, T' \xrightarrow{\text{id}_{T'}} T'$

Nella categoria Set (degli insiemi) la definizione generalizza la nozione "singleton set" (insieme contenente esattamente un unico elemento) (quindi il "singleton set" è un terminal object), in cui per ogni set X c'è precisamente una funzione $f: X \rightarrow T$ il quale mappa per ogni elemento di X sul singoletto

Ex19:

Prove that the terminal object of a category C is unique up to isomorphism.

Soluzione:

Proof:

- T è terminale per T' (morfismo $T \rightarrow T'$) e viceversa
- T, T' hanno la loro identità
- la composizione delle frecce uniche $T \rightarrow T', T' \rightarrow T$ dev'essere id_T e viceversa dall'altra parte con $\text{id}_{T'}$
- $T \xrightarrow{f} T', T' \xrightarrow{g} T, T \xrightarrow{\text{id}_T} T, T' \xrightarrow{\text{id}_{T'}} T'$

Se 1 e $1'$ sono terminali, esistono frecce uniche $1 \rightarrow 1'$ e $1' \rightarrow 1$ che sono inverse: isomorfismo

Ex20:

Let $\{a, b, c\}$ be a set with three elements. Is it terminal in the category Set? In case of a positive answer, justify the answer by means of an example. Analogously, in case of a negative answer, justify the answer by means of an example.

Soluzione:

No, perché un terminal object (T) nella categoria Set è un oggetto tale per cui per ogni oggetto X esiste un unica freccia $X \rightarrow T$ (morfismo); quindi viene generalizzata la nozione "singleton set" (insieme contenente esattamente un unico elemento)

$T = \{a, b, c\}$ non è un singolo elemento e considerando un insieme $X = \{d\}$, si avrebbe più frecce verso T (più morfismi) (violazione della definizione) es. $f: X \rightarrow T$ mappa da "d" ad "a", "b" o "c" e si avrebbe tre frecce (morfismi) diversi $f(d)=a, f(d)=b$ o $f(d)=c$

Ex21:

Recall the definition of "Product between **a** and **b**" as given in Category Theory. Then, justify why $(a, \lambda x \rightarrow x, \lambda x \rightarrow y :: b)$ cannot be taken as Product between **a** and **b**.

Soluzione:

Il prodotto cartesiano in Set è l'insieme (pattern) di $A \times B$ e con le sue proiezioni ($\pi_1: A \times B \rightarrow A, \pi_2: A \times B \rightarrow B$) (che sono morfismi) date da una tripla $(A \times B, \pi_1: A \times B \rightarrow A, \pi_2: A \times B \rightarrow B)$

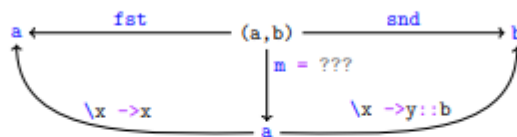
Il candidato C_a è un prodotto solo se esiste un'unica funzione ($\exists!$) $m: C_a \rightarrow A \times B$ (in categoria $(C_a, A \times B)$) che ricostruisce le proiezioni, cioè tale che $f = \pi_1 \circ m$ e $g = \pi_2 \circ m$ (fattorizza tutti gli altri

candidati)

A, B e Ca sono $\text{obj}(C)$ (oggetti della categoria)

Non è prodotto: la seconda proiezione non è ben definita universalmente e quindi non viene fattorizzata.

$(a, \lambda x. x, \lambda x. x \rightarrow y : b)$ worst than $((a, b), \text{fst}, \text{snd})$



$m = \lambda (w, _) . w$ is the unique possible choice s.t.:

- its output matches the type
- it can factorize fst
- but it cannot factorize snd

Ex22:

Recall the definition of “Product between a and b ” as given in Category Theory. Then, justify why $((a, a, b), \lambda (x, _, _) . x, \lambda (_, _, z) . z)$ cannot be taken as Product between a and b .

Soluzione:

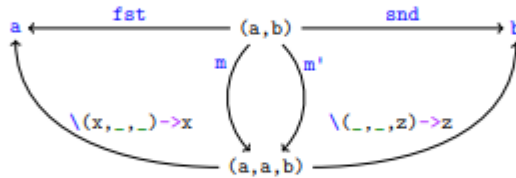
Il prodotto cartesiano in Set è l'insieme (pattern) di $A \times B$ e con le sue proiezioni $(\pi_1: A \times B \rightarrow A, \pi_2: A \times B \rightarrow B)$ (che sono morfismi) date da una tripla $(A \times B, \pi_1: A \times B \rightarrow A, \pi_2: A \times B \rightarrow B)$

Il candidato C_a è un prodotto solo se esiste un'unica funzione $(\exists!) m: C_a \rightarrow A \times B$ (in categoria $(C_a, A \times B)$) che ricostruisce le proiezioni, cioè tale che $f = \pi_1 \circ m$ e $g = \pi_2 \circ m$ (fattorizza tutti gli altri candidati)

A, B e C_a sono $\text{obj}(C)$ (oggetti della categoria)

Non è prodotto: viola l'unicità della freccia mediatrice (non esiste una freccia unica "m").

$((a, a, b), \backslash(x, _, _) \rightarrow x, \backslash(_, _, z) \rightarrow z)$ worst than $((a, b), fst, snd)$



The factorization of **fst** is not unique:

- $m = \backslash(x, y) \rightarrow (x, x, y)$
- $m' = \backslash(x, y) \rightarrow (x, n :: a, y)$, for any $n :: a$

Ex23:

Let us assume that both **P** and **Q** are the Product of **a** and **b** in a Category. Prove that **P** and **Q** are isomorphic.

Soluzione:

Quindi si hanno $P = (a, b)$ e $Q = (a, b)$ che sono oggetti della categoria $\text{obj}(C)$

$\pi_1: P \rightarrow a$ e $\pi_2: P \rightarrow b$; $\pi_1': Q \rightarrow a$ e $\pi_2': Q \rightarrow b$

Si ha quindi, la prima fase in cui si assume che **P** sia il prodotto quindi $\exists! m: Q \rightarrow P$ tale che

$\pi_1' = \pi_1 \circ m$ e $\pi_2' = \pi_2 \circ m$

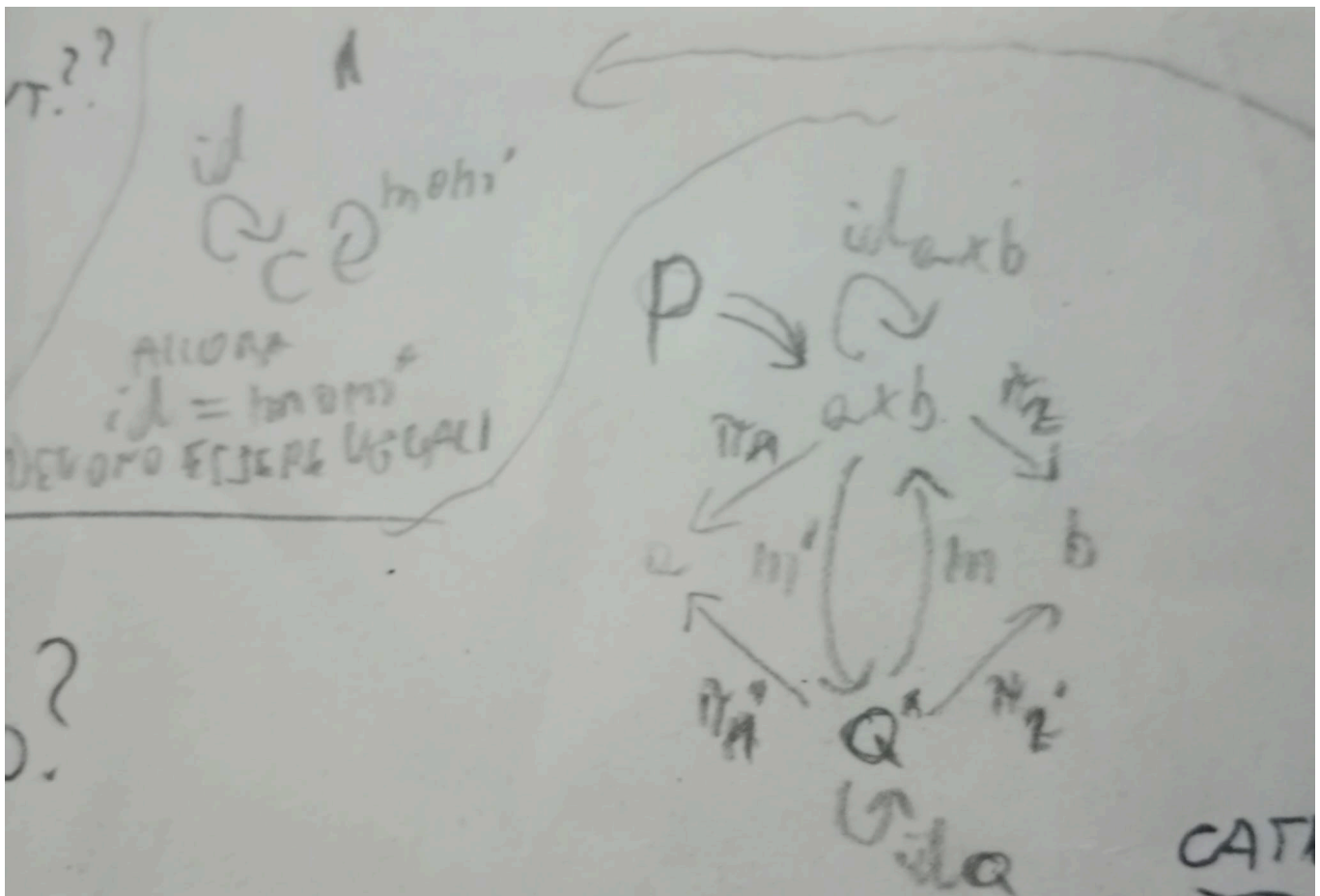
la seconda fase si assume che **Q** sia il prodotto quindi $\exists! m': P \rightarrow Q$ tale che $\pi_1 = \pi_1' \circ m'$ e

$\pi_2 = \pi_2' \circ m'$

Ora si considera la composizione data dalla proprietà universale per **P** e anche per **Q** (dell'unica freccia):

- $m \circ m' = \text{id}_P$ (si ottiene il morfismo identità tramite sostituzione algebrica di m e m' , facendo il diagramma commutativo $(P \rightarrow \text{id}_P P$ e $P \rightarrow m \circ m' P)$)
- $m' \circ m = \text{id}_Q$ (si ottiene il morfismo identità tramite sostituzione algebrica di m e m' , facendo il diagramma commutativo $(Q \rightarrow \text{id}_Q Q$ e $Q \rightarrow m' \circ m Q)$)

Dalla proprietà universale si ottengono frecce inverse uniche tra **P** e **Q**;



Ex24:

Is the following triple:

$((\text{Bool}, \text{Int}, \text{Bool}), \ \backslash (x, y, z) \rightarrow z, \ \backslash (x, y, z) \rightarrow y)$

a product between the Haskell types `Bool` and `Int`?

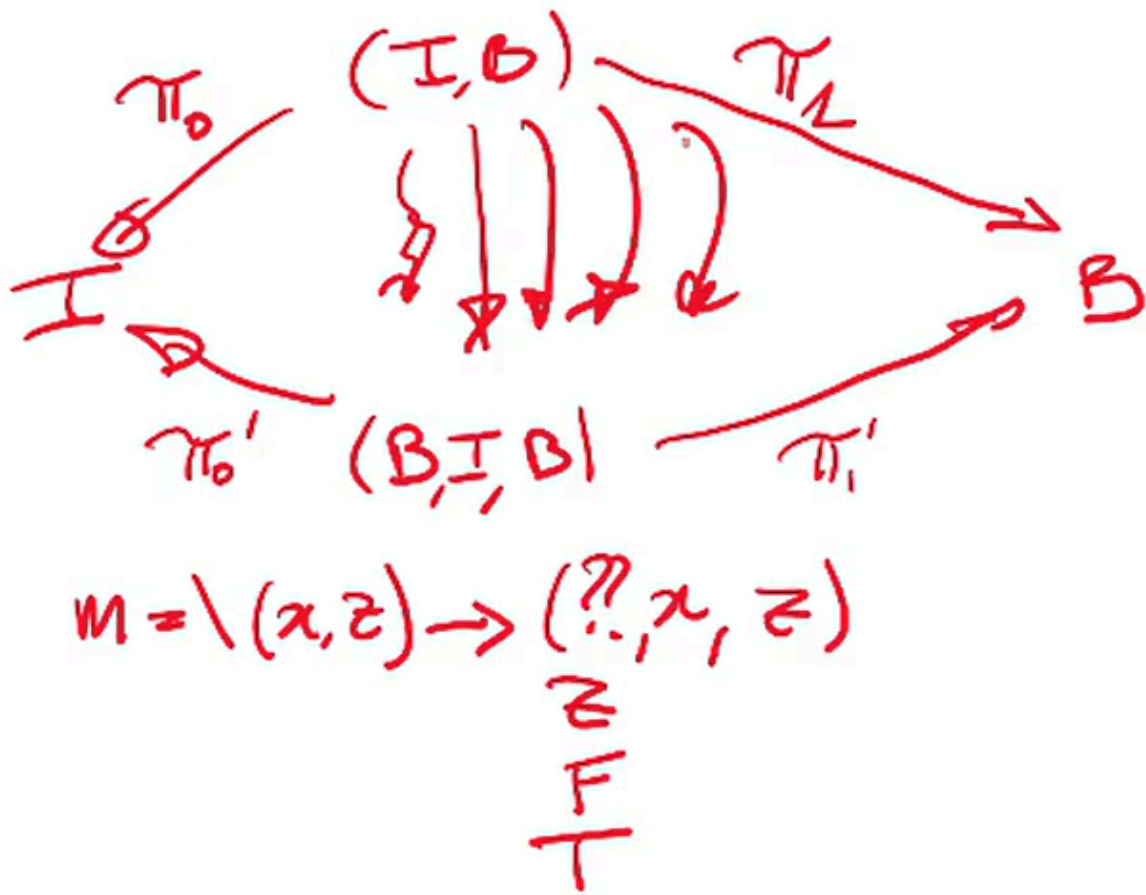
Soluzione:

π_0 e π_1 si dà per scontato i calcoli da fare, mentre $\pi'_0 = \backslash (x, y, z) \rightarrow z$ e $\pi'_1 = \backslash (x, y, z) \rightarrow y$

Non è vero che $\exists! m: (\text{Bool}, \text{Int}) \rightarrow (\text{Bool}, \text{Int}, \text{Bool})$ tale che $\pi_0 = \pi'_0 \circ m$ e $\pi_1 = \pi'_1 \circ m$

Non è prodotto: viola l'unicità della freccia mediatrice (non esiste una freccia unica "m") (Non è

unique arrow)



Ex25:

Is the following triple:

$((\text{Bool}, \text{Int}, \text{Bool}),$

$\lambda (x, y, z) \rightarrow x,$

$\lambda (x, y, z) \rightarrow y)$

a product between the Haskell types `Bool` and `Int`?

Soluzione:

No, stessa spiegazione del ex24

Ex26:

Is the following triple:

$((\text{Bool}, \text{Int}),$

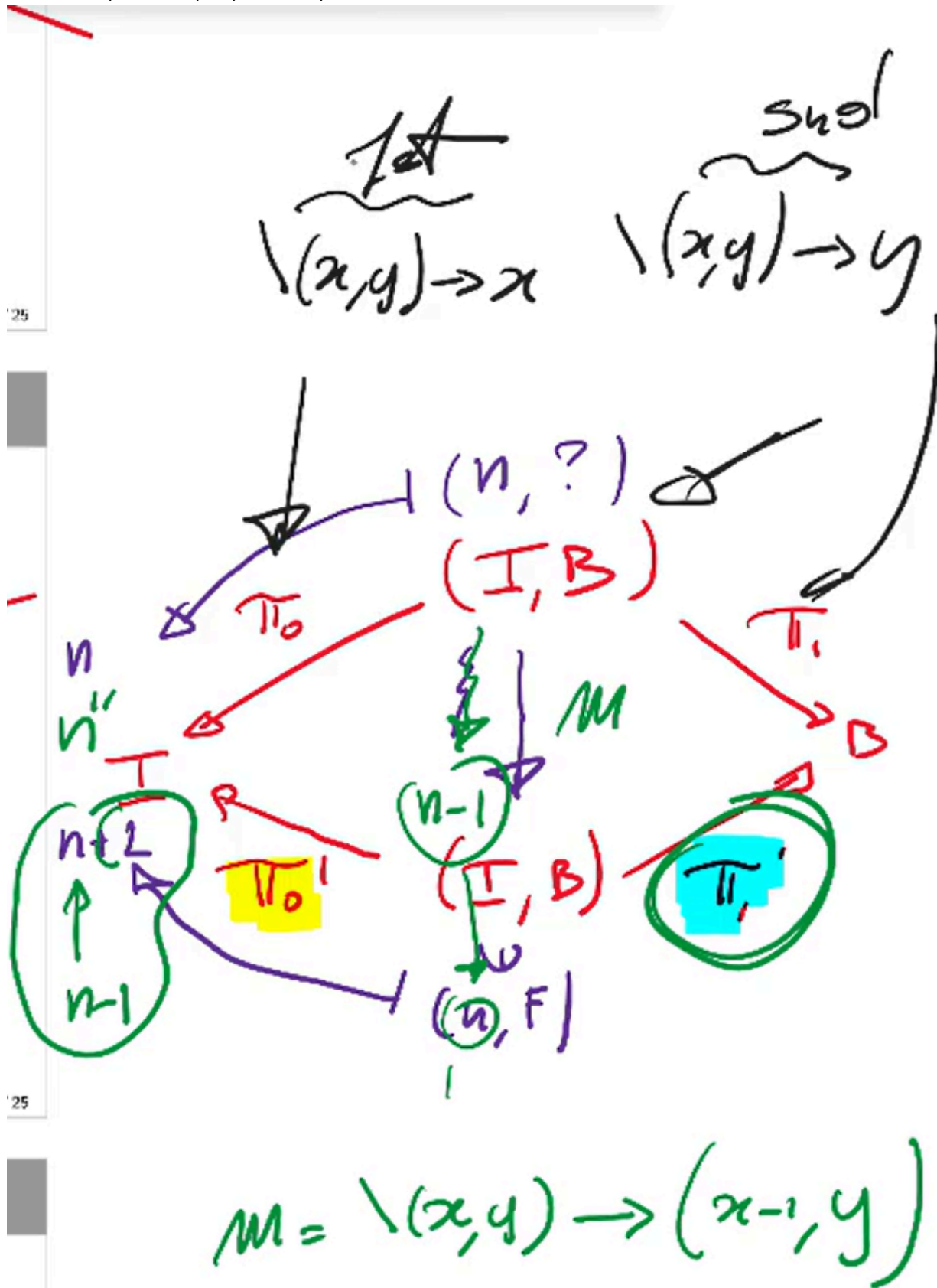
$\lambda (x, y) \rightarrow x,$

$\lambda (x, y) \rightarrow y + 1)$

a product between the Haskell types `Bool` and `Int`?

Soluzione:

Si, $\exists! m: (\text{Bool}, \text{Int}) \rightarrow (\text{Bool}, \text{Int})$ tale che $\pi_0 = \pi_0' \circ m$ e $\pi_1 = \pi_1' \circ m$



Ex27:

Is the following triple:

$((\text{Bool}, \text{Int}),$

$\backslash (x, y) \rightarrow \text{not } x,$

$\backslash (x, y) \rightarrow y)$

a product between the Haskell types `Bool` and `Int`?

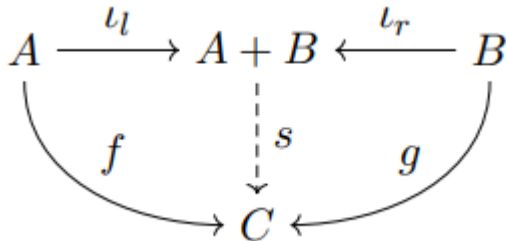
Soluzione:

Sì, simile al ex26, in cui $\exists! m: (\text{Bool}, \text{Int}) \rightarrow (\text{Bool}, \text{Int})$ tale che $\pi_0 = \pi_0' \circ m$ e $\pi_1 = \pi_1' \circ m$

in cui $m = \backslash (x, y) \rightarrow (\text{not } x, y)$

CT0200 Universal constructions

In una categoria il co-product di due oggetti A e B è dato da una tripla $(A+B, \iota_l, \iota_r)$ tale che il: $A \rightarrow A+B$ e $\iota_r: B \rightarrow A+B$; per ogni oggetto C e ogni coppia di freccia $f: A \rightarrow C, g: B \rightarrow C$ esiste un'unica freccia $s: A+B \rightarrow C$ che rende commutativo il diagramma $f = s \circ \iota_l, g = s \circ \iota_r$



Il pattern del co-product è:

- unisce due oggetti
- mantiene traccia dell'origine degli elementi
- tramite etichette (Left/Right)

In `Set` $A+B$ è l'unione disgiunta di A e B

In Haskell data `Either a b = Left a | Right b`

- `Left :: a -> Either a b` (iniezione sinistra)
- `Right :: b -> Either a b` (iniezione destra)

`Either a b` è il co-product perché:

- più informativo di ogni altro candidato (non fattorizzazione),
- tale che ogni altro schema si fattorizzi unicamente attraverso di esso (non unique, viola l'universalità).

Un candidato $(C_a, \text{Left}', \text{Right}')$ dove C_a è il candidato (cioè un qualsiasi oggetto che pretende di essere “unito insieme” da A e B ed è dotato di due frecce) mentre $\text{Left}', \text{Right}'$ sono le sue proiezioni che permettono di collegare A e B con C_a

Il candidato C_a è un co-prodotto solo se esiste un'unica funzione $s: A+B \rightarrow C_a$ (in categoria

$(A+B, Ca)$ che ricostruisce le proiezioni, cioè tale che $Left' = s \circ Left$ e $Right' = s \circ Right$ (fattorizza tutti gli altri candidati)

Il co-product come costruzione universale, è l'oggetto che rappresenta nel modo migliore possibile tutte le coppie di frecce uscenti da A e B. Inoltre è unico a meno di isomorfismo, ogni altro co-prodotto è isomorfo a $A+B$

$A+B == \text{Either } A \ B$

Ex28:

Is the following triple:

$(\text{Either } (\text{Either Bool Int}) \text{ Int}$

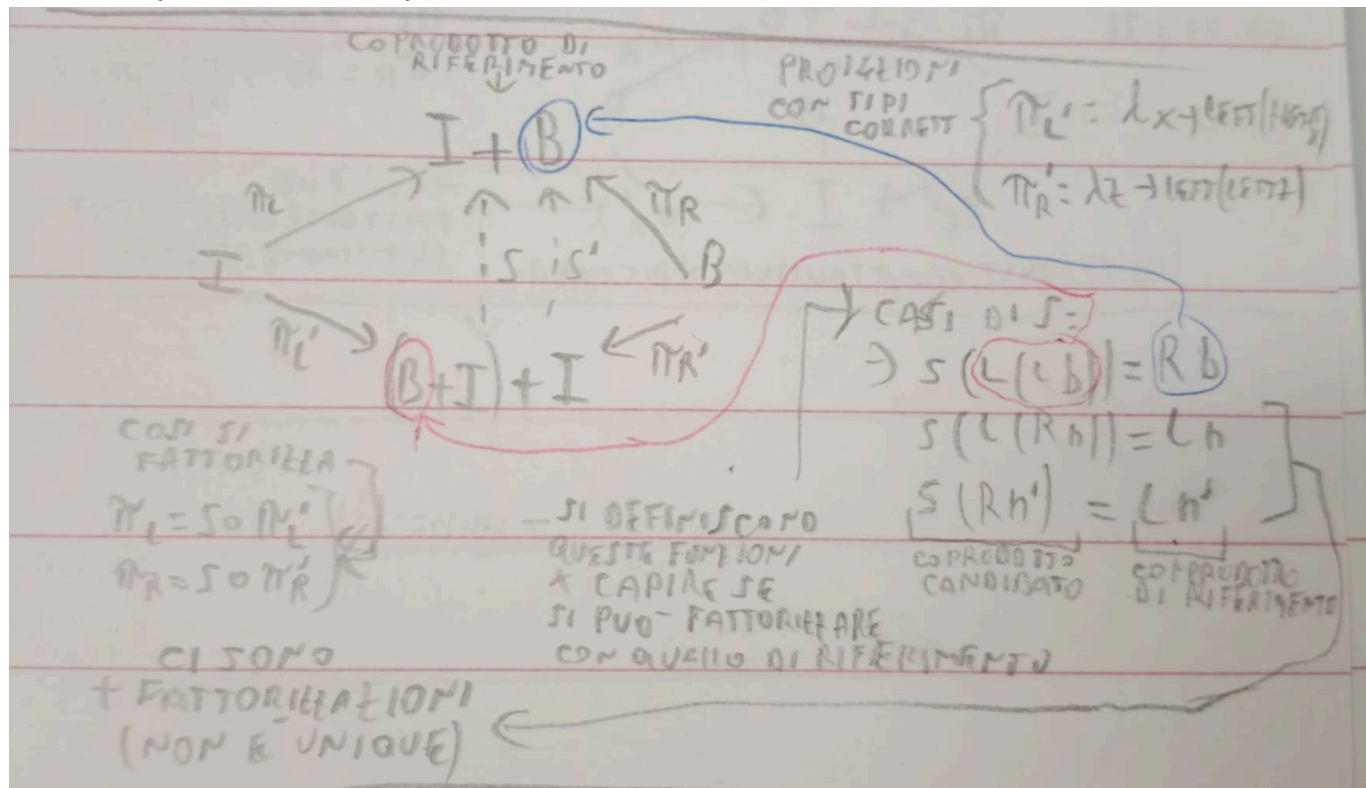
$, \lambda x :: \text{Int} \rightarrow \text{Left } (\text{Right } x)$

$, \lambda z :: \text{Bool} \rightarrow \text{Left } (\text{Left } z))$

a Co-Product between Int and Bool?

Soluzione:

Non è unique, struttura con più fattorizzazioni



Ex29:

Is the following triple:

$(\text{Either Int Bool}$

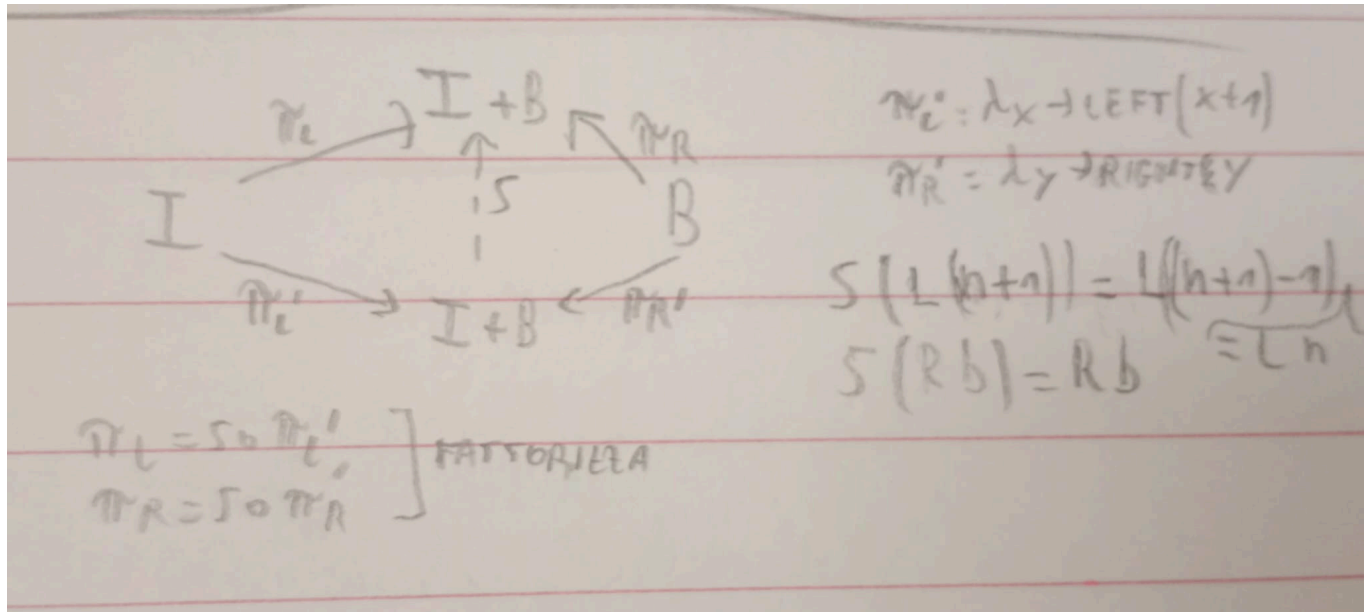
$, \lambda x :: \text{Int} \rightarrow \text{Left } (x + 1)$

,y::Bool -> Right y)

a Co-Product between Int and Bool?

Soluzioni:

Si è un co-product in cui $\exists ! s: \text{Int} + \text{Bool} \rightarrow \text{Int} + \text{Bool}$ tale che $\pi_l = s \circ \pi_l'$ e $\pi_r = s \circ \pi_r'$



Ex30:

Is the following triple:

(Either Int Bool

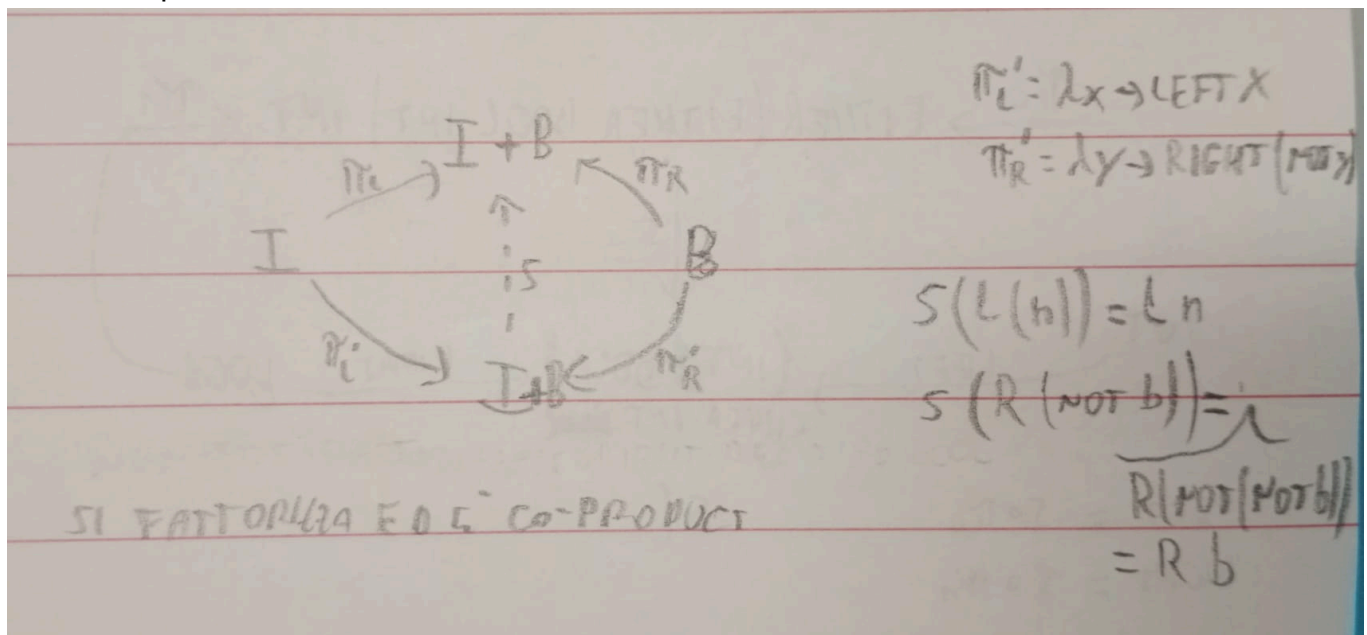
,x::Int -> Left x

,y::Bool -> Right (not y))

a Co-Product between Int and Bool?

Soluzione:

Si è un co-product in cui $\exists ! s: \text{Int} + \text{Bool} \rightarrow \text{Int} + \text{Bool}$ tale che $\pi_l = s \circ \pi_l'$ e $\pi_r = s \circ \pi_r'$



Ex31:

Is the following triple:

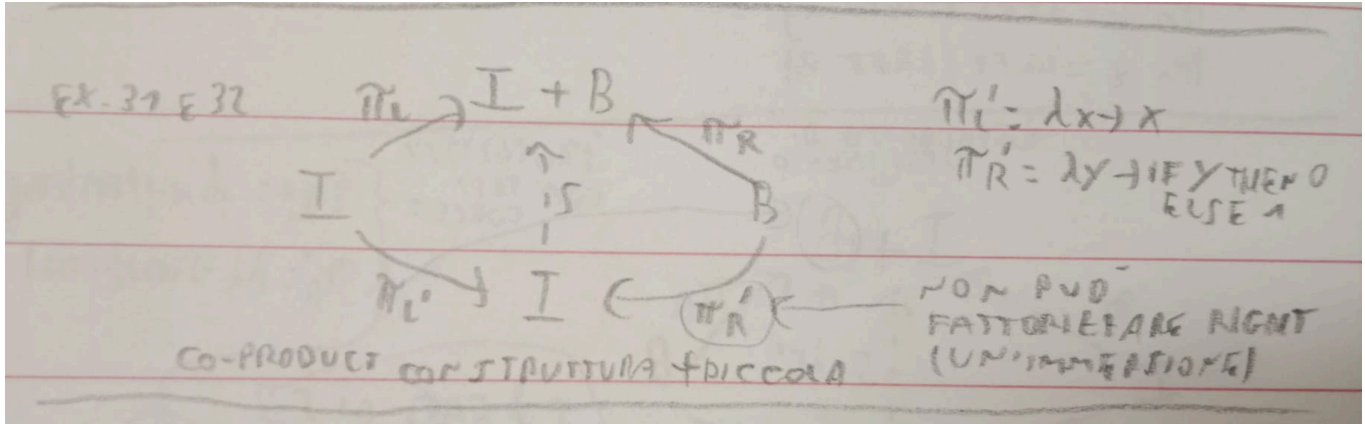
$(\text{Int}, \lambda x::\text{Int} \rightarrow x$

$, \lambda y::\text{Bool} \rightarrow \text{if } y \text{ then } 0 \text{ else } 1)$

a Co-Product between Int and Bool?

Soluzione:

non puoi distinguere elementi di B manca informazione, per cui fallisce l'esistenza di s



Ex32:

Is the following triple:

$(\text{Int}, \lambda x::\text{Int} \rightarrow \text{if } x < 0 \text{ then } x \text{ else } x+2$

$, \lambda y::\text{Bool} \rightarrow \text{if } y \text{ then } 0 \text{ else } 1)$

a Co-Product between Int and Bool?

Soluzione:

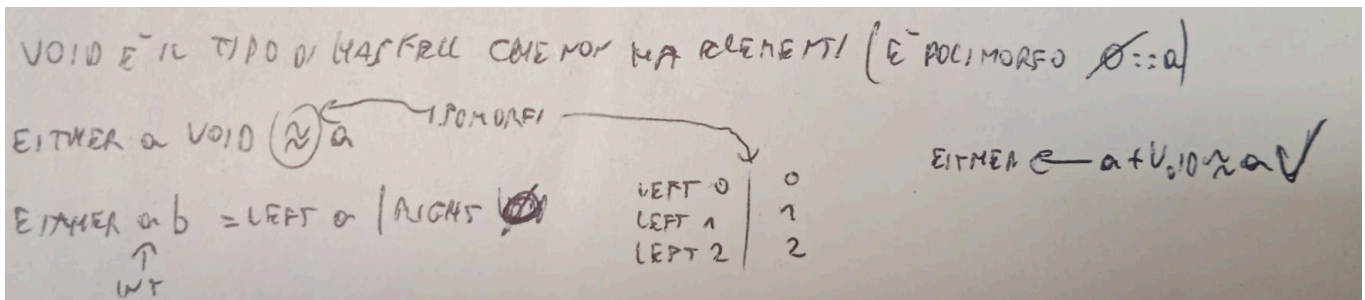
Vale lo stesso per ex31

Ex33:

Justify why "Either a Void" and a are isomorphic.

Soluzione:

è isomorfo, si ha una freccia e la sua inversa che danno lo stesso morfismo di identità

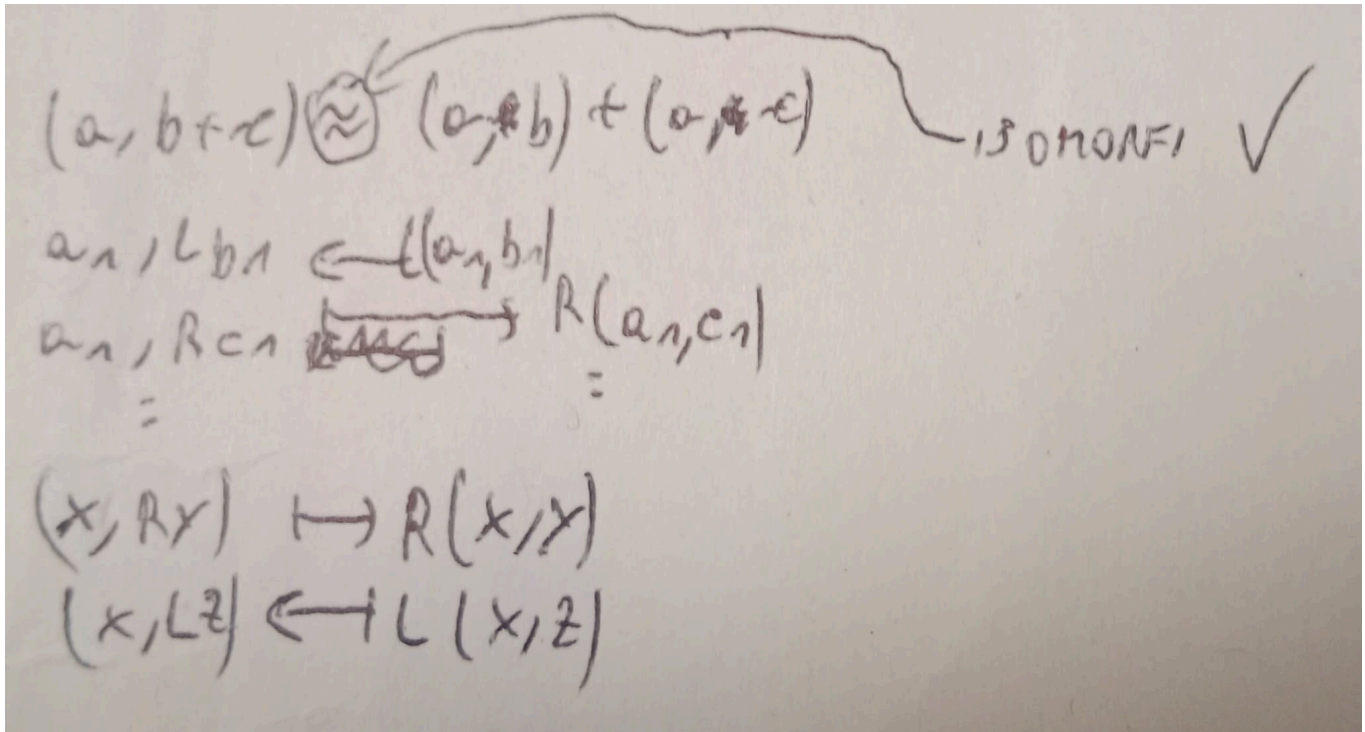


Ex34:

Justify why “(a, Either b c)” and “Either (a, b) (a, c)” are isomorphic.

Soluzione:

è isomorfo, si ha una freccia e la sua inversa che danno lo stesso morfismo di identità



Ex35:

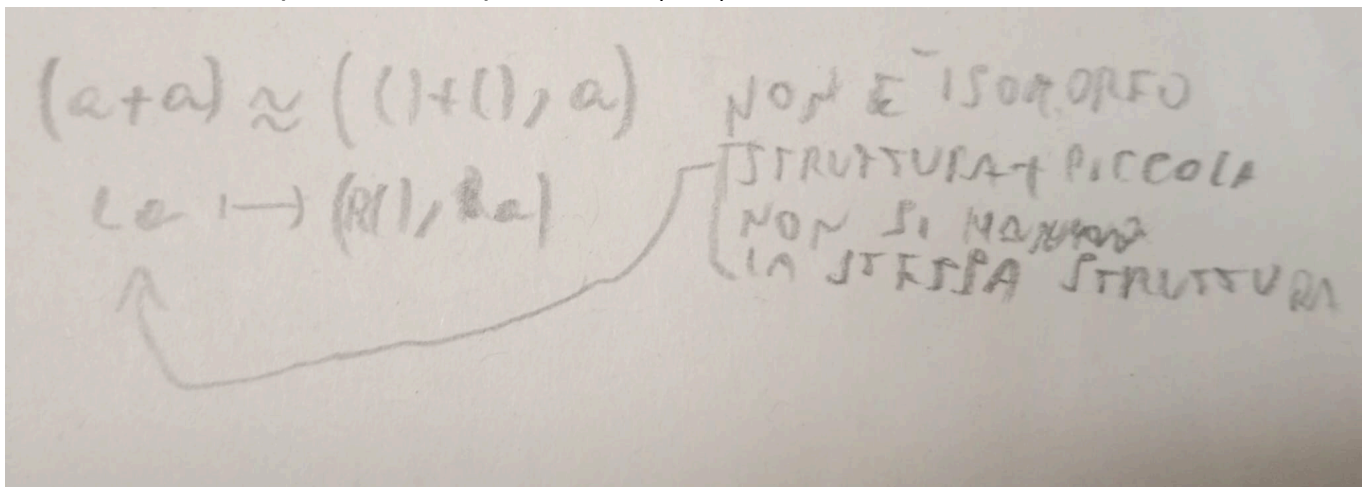
Are “Either a a” and “(Either () (), a)” isomorphic?

Soluzione:

L'espressione "Either a a" rappresenta un tipo di dato che può essere una delle due istanze di 'a'. La sua struttura è equivalente a $a+a$.

L'espressione "(Either () (), a)" rappresenta un tipo di dato che è una coppia (prodotto cartesiano) dove il primo elemento è un "Either () ()" e il secondo elemento è 'a'. L'espressione interna "Either () ()" rappresenta $1+1$, che è equivalente al tipo booleano o al tipo unitario.

Pertanto, l'intera espressione è equivalente a $(1+1) \times a$.



CT0300 Functors

Un functor, nella teoria delle categorie, è una mappa tra categorie (funzioni che associano gli oggetti e i morfismi di una categoria a quelli di un'altra) che preserva oggetti e frecce (la struttura fondamentale)

In Haskell è un type constructor F dotato di $\text{fmap} :: (a \rightarrow b) \rightarrow F a \rightarrow F b$ che soddisfa:

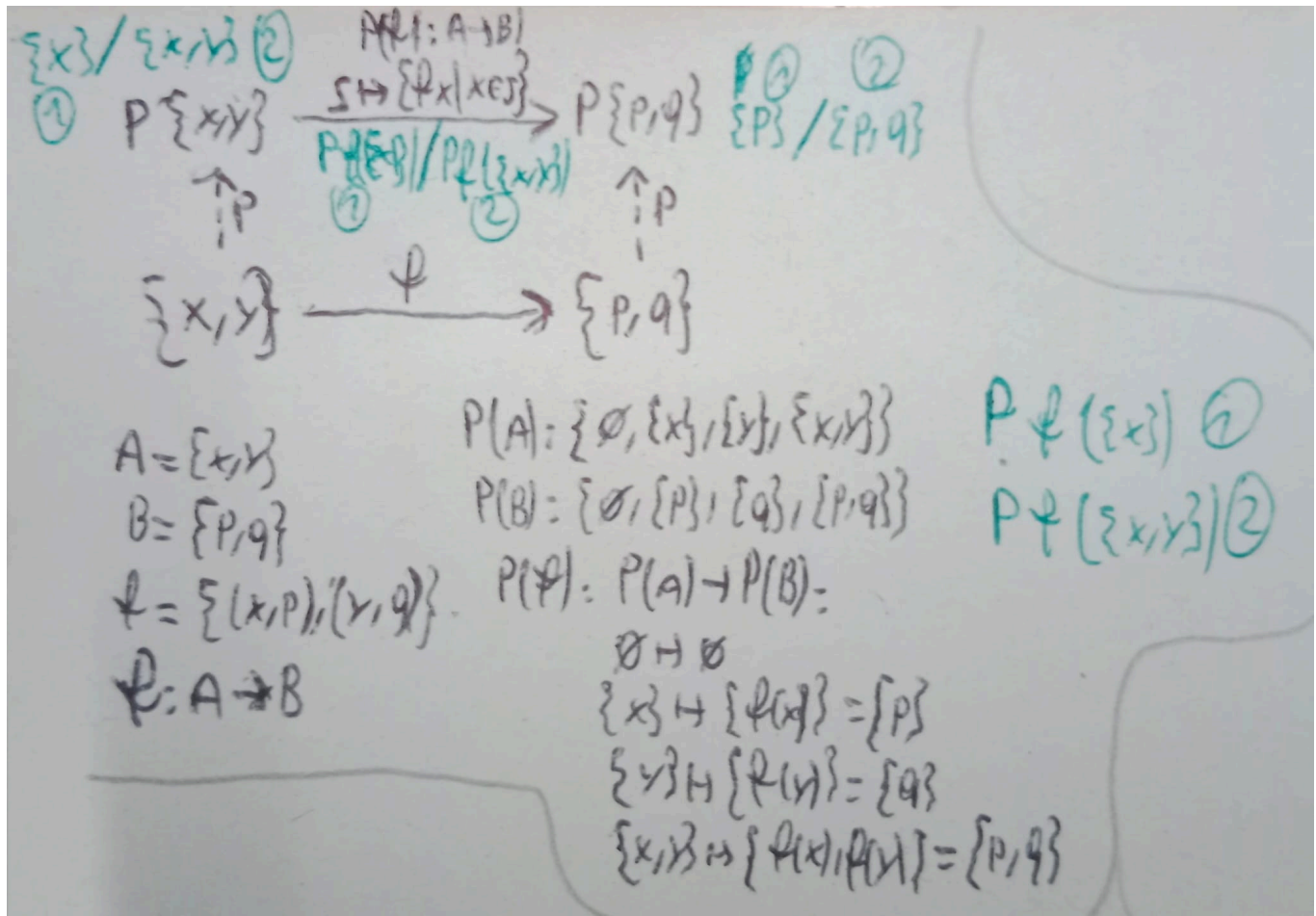
1. $\text{fmap id} = \text{id}$ (legge di identità)
2. $\text{fmap } (g.f) = \text{fmap } g \circ \text{fmap } f$ (legge di composizione)

Ex1:

Let $A = \{x, y\}$ and $B = \{p, q\}$ be sets. Let $f = \{(x, p), (y, q)\}$ be a function on sets.

- Draw the diagram resulting from the application of the "Powerset" functor to A , B , and f .
- Decorate the diagram with the elements $P f(\{x\})$ and $P f(\{x, y\})$

Soluzione:

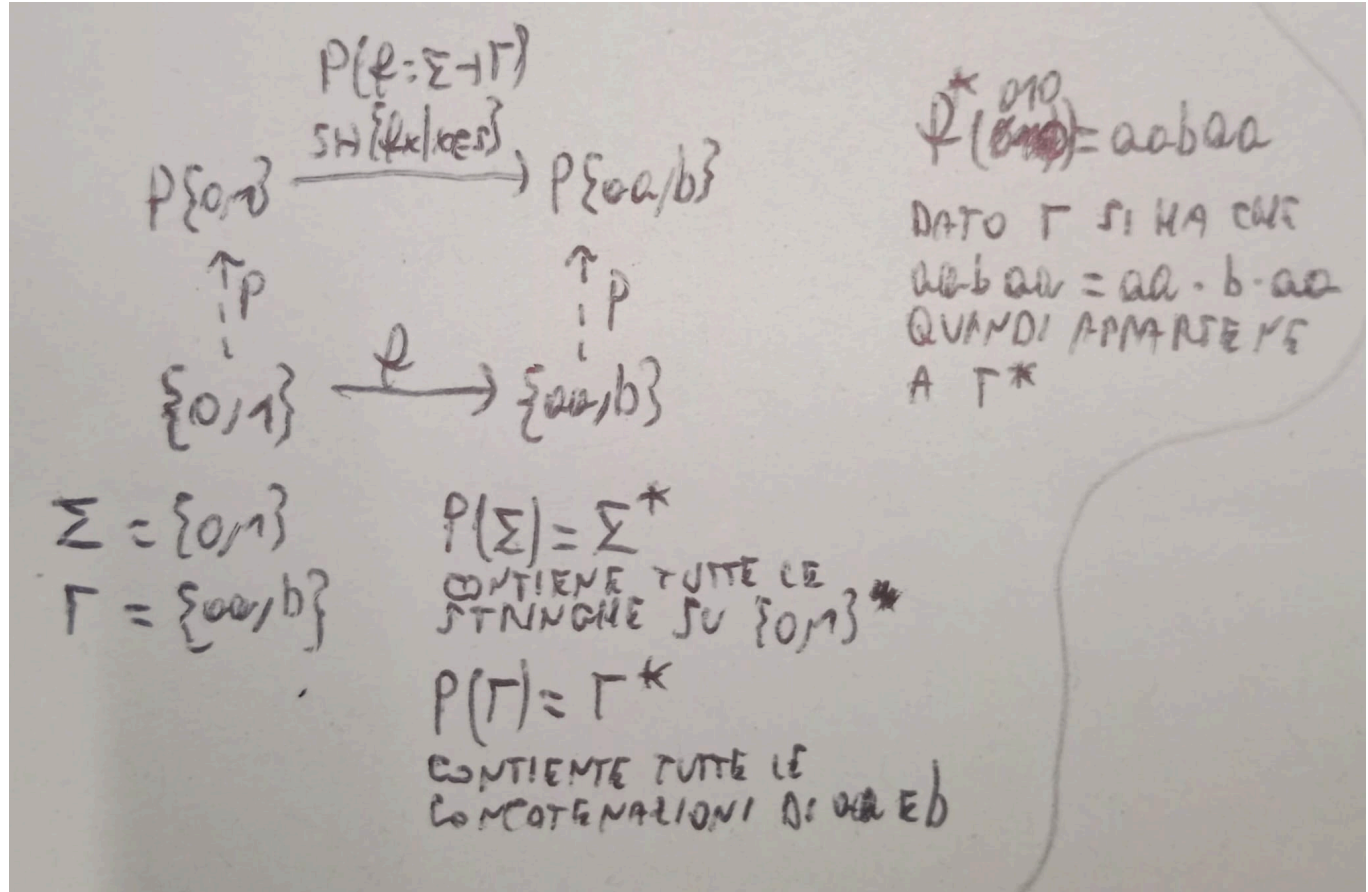


Ex2:

Let $\Sigma = \{0, 1\}$, $\Gamma = \{aa, b\}$ be two alphabets. Let $f(0) = aa$, $f(1) = b$ be two functions between alphabets.

- Draw the diagram resulting from the application of the Kleene Star functor to Σ , Γ , and f .
- Can $aabaa$ be an element of Γ^* generated by the Kleene Star functor $(-)^*$? Justify the answer.

Soluzione:



Ex3:

Let $\Sigma = \{x, y, z\}$, $\Gamma = \{p, qq, r\}$ be two alphabets. Let $f(x) = p$, $f(y) = qq$, $f(z) = r$ be two functions between alphabets.

- Draw the diagram resulting from the application of the Kleene Star functor to Σ , Γ , and f .
- Is $pqqrqqp$ an element of Γ^* obtained via the Kleene Star construction? Explain your reasoning.

Soluzione:

Come l'Ex2

$$pqqrqqp = p \cdot qq \cdot r \cdot qq \cdot p$$

Corrisponde a $xyz zx$. Quindi appartiene a Γ^*

Ex4:

Let $\Sigma = \{u, v\}$, $\Gamma = \{mm, n\}$ be alphabets. Let $f(u) = mm$, $f(v) = n$ be functions between alphabets.

- Draw the diagram resulting from the application of the Kleene Star functor to Σ , Γ , and f .
- Determine whether $mmnmmn$ belongs to Γ *under the Kleene Star functor* $(-)^*$. Provide justification.

Soluzione:

Come l'Ex2

$mmnmmn = mm \cdot n \cdot mm \cdot n$

Corrisponde a $uvuv$. Quindi appartiene a Γ^*

Ex5:

Recall the definition of the data-type "Maybe a". Prove that it is a functor by:

1. defining a function $fmap$;
2. proving that $fmap$ enjoys the identity law;
3. proving that $fmap$ enjoys the composition law.

Soluzione:

Il tipo "Maybe a" è definito come: $\text{data Maybe } a = \text{Nothing} \mid \text{Just } a$

1. La funzione $fmap$ per "Maybe" è definito come: $fmap :: (a \rightarrow b) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b$
 - $fmap \ f \ \text{Nothing} = \text{Nothing}$
 - $fmap \ f \ (\text{Just } x) = \text{Just } (f \ x)$
 - dove f è una funzione generale tale che $f: a \rightarrow b$
2. La legge di identità di $fmap$ afferma che $fmap \ id = id$ (quindi non modifica la struttura)
 - $fmap \ id \ \text{Nothing} = \text{Nothing} = id \ \text{Nothing}$
 - $fmap \ id \ (\text{Just } x) = \text{Just } (id \ x) = \text{Just } x = id \ (\text{Just } x)$
3. La legge di composizione di $fmap$ afferma che $fmap \ (f.g) = fmap \ f \ . \ fmap \ g$
 - Caso Nothing:
 - Sinistra: $fmap \ (f.g) \ \text{Nothing} = \text{Nothing}$
 - Destra: $(fmap \ f \ . \ fmap \ g) \ \text{Nothing} = fmap \ f \ (fmap \ g \ \text{Nothing}) = fmap \ f \ \text{Nothing} = \text{Nothing}$
 - Caso Just x:
 - Sinistra: $fmap \ (f.g) \ \text{Just } x = \text{Just } ((f.g) \ x) = \text{Just } (f(g \ x))$
 - Destra: $(fmap \ f \ . \ fmap \ g) \ \text{Just } x = fmap \ f \ (fmap \ g \ (\text{Just } x)) = fmap \ f \ (\text{Just } (g \ x)) = \text{Just } (f \ (g \ x))$

Ex6:

Define a data-type "List a" that recursively formalizes the corresponding concept. Then, prove that it is a functor by:

1. defining a function fmap;
2. proving that fmap enjoys the identity law;
3. proving that fmap enjoys the composition law.

Soluzione:

Il tipo "List a" è definito come: $\text{data List a} = \text{Nil} \mid \text{Cons a (List a)}$

Ricorda: Cons a (List a) : aggiunge un elemento di tipo a a una lista esistente (passo ricorsivo).

1. La funzione fmap per "List" è definito come: $\text{fmap} :: (a \rightarrow b) \rightarrow \text{List a} \rightarrow \text{List b}$
 - $\text{fmap f Nil} = \text{Nil}$
 - $\text{fmap f (Cons x xs)} = \text{Cons (f x) (fmap f xs)}$
 - dove f è una funzione generale tale che $f: a \rightarrow b$
2. La legge di identità di fmap afferma che $\text{fmap id} = \text{id}$ (quindi non modifica la struttura)
 - $\text{fmap id Nil} = \text{Nil} = \text{id Nil}$
 - $\text{fmap id (Cons x xs)} = \text{Cons (id x) (fmap id xs)} = \text{Cons (x xs)} = \text{id Cons (x xs)}$
3. La legge di composizione di fmap afferma che $\text{fmap (f.g)} = \text{fmap f} . \text{fmap g}$
 - Caso Nil:
 - Sinistra: $\text{fmap (f.g) Nil} = \text{Nil}$
 - Destra: $(\text{fmap f} . \text{fmap g}) \text{Nil} = \text{fmap f (fmap g Nil)} = \text{fmap f Nil} = \text{Nil}$
 - Caso Cons x xs:
 - Sinistra: $\text{fmap (f.g) Cons x xs} = \text{Cons (f.g x) (fmap f.g xs)} = \text{Cons (f(g(x))) (fmap f(g(xs)))}$ $= (\text{H.P.}) \text{Cons (f(g(x))) (fmap f (fmap (g xs)))}$ $= (\text{DEF. di fmap f funzione più esterna}) \text{fmap f (Cons (g x) (fmap g xs))}$
 - Destra: $(\text{fmap f} . \text{fmap g}) \text{Cons x xs} = \text{fmap f (fmap g (Cons x xs))} = \text{fmap f (Cons (g x) (fmap g xs))}$

Ex7:

Recall the definition of the data-type "Either a b" and prove that "Either a" is a functor by:

1. defining a function fmap;
2. proving that fmap enjoys the identity law;
3. proving that fmap enjoys the composition law.

Soluzione:

Il tipo "Either a b" è definito come: $\text{data Either a b} = \text{Left a} \mid \text{Right b}$, si prova che "Either a" è un Functor quindi fmap agisce sul tipo b

1. La funzione fmap per "Either a" è definito come: $\text{fmap} :: (b \rightarrow c) \rightarrow \text{Either a b} \rightarrow \text{Either a c}$
 - $\text{fmap f (Left x)} = \text{Left x}$
 - $\text{fmap f (Right y)} = \text{Right (f y)}$

- dove f è una funzione generale tale che $f: a \rightarrow b$
2. La legge di identità di fmap afferma che $\text{fmap id} = \text{id}$ (quindi non modifica la struttura)
 - $\text{fmap id (Left x)} = \text{Left x} = \text{id Left x}$
 - $\text{fmap id (Right y)} = \text{Right (id y)} = \text{Right (y)} = \text{id Right (y)}$
 3. La legge di composizione di fmap afferma che $\text{fmap (f.g)} = \text{fmap f} . \text{fmap g}$
 - Caso Left x:
 - Sinistra: $\text{fmap (f(g(Left x)))} = \text{Left x}$
 - Destra: $\text{fmap f (fmap g (Left x))} = \text{fmap f (Left x)} = \text{Left x}$
 - Caso (Right y):
 - Sinistra: $\text{fmap (f(g(Right y)))} = \text{Right (f(g(y)))}$
 - Destra: $\text{fmap f (fmap g (Right y))} = \text{fmap f (Right (g y))} = \text{Right (f (g y))}$

Ex8:

Consider the following Haskell data-type:

`data BTree l where`

`Leaf :: l -> BTree n l`

`Node :: BTree n l -> BTree n l -> BTree n l`

1. Define the function `fmap`.
2. Prove that `fmap` enjoys the identity law.

Soluzione:

Attenzione (possibile errore) si definisce il tipo come `BTree l` oppure come `BTree n l`; il problema è dovuto al fatto che nei costruttori usano `BTree n l` come valori di ritorno, quindi in quel caso è definito come `BTree n l`, altrimenti bisogna cambiari i costruttori con il valore di ritorno `BTree l`

Io definirò la `fmap` come `BTree l`

Il data-type di `BTree` viene definito come `data BTree l = Leaf l | Node (BTree l) (BTree l)`

1. La funzione `fmap` per "BTree" è definito come: `fmap :: (a->b) -> BTree a -> BTree b`
 - $\text{fmap f (Leaf x)} = \text{Leaf (f x)}$
 - $\text{fmap f (Node l r)} = \text{Node (fmap f l) (fmap f r)}$
 - dove l r indicano il sottoalbero sinistro e quello destro nel pattern matching del `Node`
2. La legge di identità di `fmap` afferma che `fmap id = id` (quindi non modifica la struttura)
 - Caso base: `Leaf x`
 - $\text{fmap id (Left x)} = \text{Left (id x)} = \text{Left x} = \text{id Left x}$
 - Caso Induttivo: `Node l r`

- $\text{fmap id (Node l r)} = \text{Node (fmap id l) (fmap id r)} = \text{Node (id l) (id r)} = \text{Node l r} = \text{id (Node l r)}$

Ex9:

Consider the following Haskell data-type:

`data BTree l where`

`Leaf :: l -> BTree n l`

`Node :: BTree n l -> BTree n l -> BTree n l`

1. Define the function `fmap`.
2. Prove that `fmap` enjoys the composition law.

Soluzione:

Stessi problemi e stessi discorsi dell'esercizio precedente, ma con l'aggiunta della composizione

2) La legge di composizione di `fmap` afferma che $\text{fmap (f.g)} = \text{fmap f} . \text{fmap g}$; si procede per induzione sulla struttura dell'albero (essendo che si parla di `BTree`)

- Caso base: `Leaf x`
- Sinistra: $\text{fmap (f(g(Leaf x)))} = \text{Left (f(g(x)))}$
- Destra: $\text{fmap f (fmap g (Leaf x))} = \text{fmap f (Leaf (g x))} = \text{Leaf (f(g(x)))}$
- Caso Induttivo: `Node l r`
- Sinistra: $\text{fmap (f (g (Node l r)))} = \text{Node (fmap f (g l)) (fmap f (g r))} = (\text{H.P.}) \text{Node (fmap f (fmap g l)) (fmap f (fmap g r))} = \text{fmap f (Node (fmap g l) (fmap g r))}$
- Destra: $\text{fmap f (fmap g (Node l r))} = \text{fmap f (Node (fmap g l) (fmap g r))}$

Ex10:

Consider the following Haskell data-type:

`data BTree n where`

`Leaf :: BTree n`

`Node :: n -> BTree n -> BTree n -> BTree n`

1. Define the function `fmap`.
2. Prove that `fmap` enjoys the identity law.

Soluzione:

Il data-type di "`BTree n`" viene definito come `data BTree n = Leaf | Node n (BTree n) (BTree n)`

1. La funzione `fmap` per "`BTree`" è definito come: $\text{fmap} :: (a \rightarrow b) \rightarrow \text{BTree a} \rightarrow \text{BTree b}$
 - $\text{fmap f (Leaf)} = \text{Leaf}$
 - $\text{fmap f (Node x l r)} = \text{Node (f x) (fmap f l) (fmap f r)}$
2. La legge di identità di `fmap` afferma che $\text{fmap id} = \text{id}$ (quindi non modifica la struttura)

- Caso base: Leaf
 - $\text{fmap id (Left)} = \text{Left} = \text{id Left}$
- Caso Induttivo: Node x l r
 - $\text{fmap id (Node x l r)} = \text{Node (id x) (fmap id l) (fmap id r)} = \text{Node (id x) (id l) (id r)} = \text{Node x l r} = \text{id (Node x l r)}$

Ex11:

Consider the following Haskell data-type:

data BTree n where

Leaf :: BTree n

Node :: n -> BTree n -> BTree n -> BTree n

1. Define the function fmap.
2. Prove that fmap enjoys the composition law.

Soluzione:

Stessi discorsi dell'esercizio precedente, ma con la legge della composizione

2) La legge della composizione afferma che $\text{fmap (f . g)} = \text{fmap f . fmap g}$

- Caso base: Leaf
- Sinistra: $\text{fmap f (g (Leaf))} = \text{Leaf}$
- Destra: $\text{fmap f (fmap g (Leaf))} = \text{fmap f (Leaf)} = \text{Leaf}$
- Caso Induttivo: Node x l r
 - Sinistra: $\text{fmap f (g (Node x l r))} = \text{Node (f(g(x))) (fmap f(g(l))) (fmap f(g(r)))} = (\text{H.P.}) \text{Node (f(g(x))) (fmap f(fmap g(l))) (fmap f(fmap g(r)))}$
 - Destra: $\text{fmap f (fmap g (Node x l r))} = \text{fmap f (Node (g(x)) (fmap g(l)) (fmap g(r)))} = \text{Node (f(g(x))) (fmap f(fmap g(l))) (fmap f (fmap g(r)))}$

Ex12:

Consider the following Haskell data-type:

data T a where

TL :: a -> T a

TN :: [T a] -> T a

1. Define the function fmap.
2. Prove that fmap enjoys the identity law.

Soluzione:

Il data-type di "T a" viene definito come $\text{data T a} = \text{TL a} \mid \text{TN [T a]}$

1. La funzione fmap per "T" è definito come: $\text{fmap} :: (a \rightarrow b) \rightarrow T a \rightarrow T b$
 - $\text{fmap f (TL x)} = \text{TL (f x)}$

- $\text{fmap } f \text{ (TN } xs) = \text{TN (map (fmap } f) xs)$
 - N.B.
 - Il costruttore `TN` contiene una lista di sotto-alberi `[T a]`
 - Dobbiamo trasformare ogni elemento della lista da `T a` a `T b`.
 - Poiché gli elementi della lista sono a loro volta oggetti di tipo `T`, dobbiamo chiamare `fmap f` ricorsivamente su ciascuno di essi.
 - Usiamo la funzione standard `map` delle liste per applicare la nostra funzione ricorsiva `(fmap f)` a ogni sotto-albero nella lista `xs`
 - Infine, ri assembliamo i risultati in un nuovo nodo `TN`

2. La legge di identità di `fmap` afferma che `fmap id = id` (quindi non modifica la struttura)

- Caso base: `TL x`
 - $\text{fmap id (TL } x) = \text{TL (id } x) = \text{TL } x = \text{id (TL } x)$
- Caso Induttivo: `TN xs`
 - $\text{fmap id (TN } xs) = \text{TN (map (fmap id) } xs) = \text{TN (map id } xs) = \text{TN } xs = \text{id (TN } xs)$

Ex13:

Consider the following Haskell data-type:

`data T a where`

`TL :: a -> T a`

`TN :: [ T a ] -> T a`

1. Define the function `fmap`.
2. Prove that `fmap` enjoys the composition law.

Soluzione:

Stessi discorsi dell'esercizio precedente, ma con la legge della composizione

2) La legge della composizione di `fmap` afferma che $\text{fmap (f . g)} = \text{fmap } f . \text{fmap } g$

- Caso Base: `TL x`

- Sinistra: $\text{fmap (f . g) (TL } x) = \text{TL (f (g(x)))}$

- Destra: $\text{fmap } f . \text{fmap } g \text{ (TL } x) = \text{fmap } f \text{ (TL (g(x)))} = \text{TL (f (g(x)))}$

- Caso Induttivo: `TN xs`

- Sinistra: $\text{fmap (f . g) (TN } xs) = \text{TN (map (fmap (f . g)) } xs) \stackrel{\text{(H.P.)}}{=} \text{TN (map (fmap } f . \text{fmap } g) } xs)$

- Destra: $\text{fmap } f . \text{fmap } g \text{ (TN } xs) = \text{fmap } f \text{ (TN (map (fmap } g) } xs)) = \text{TN (map (fmap } f) \text{ (map (fmap } g) } xs)) = \text{TN (map (fmap } f) . \text{(map (fmap } g) } xs))$

- ATTENZIONE si usa la proprietà della funzione `map` : $\text{map (f . g)} = \text{map } f . \text{map } g$; dove mappare la composizione di due funzioni `f` e `g` su una lista è equivalente a mappare `g` e poi mappare `f` sulla lista risultante

- $\text{TN (map (fmap } f . \text{fmap } g) } xs) \stackrel{\text{(Sx)}}{=} \stackrel{\text{(Dx)}}{=} \text{TN (map (fmap } f) . \text{map (fmap } g) } xs)$

Ex14:

Consider the following Haskell data-type:

data U a b where

UL:: Either a b -> U a b

UN:: (U a b, U a b) -> U a b

1. Define the function fmap.
2. Prove that fmap enjoys the identity law.

Soluzione:

Il data-type di "U a b" viene definito come $\text{data U a b} = \text{UL (Either a b)} \mid \text{UN (U a b, U a b)}$; si prova che "U a" è un Functor quindi fmap agisce sul tipo b

1. La funzione fmap per "U a" è definito come: $\text{fmap} :: (b \rightarrow c) \rightarrow \text{U a b} \rightarrow \text{U a c}$
 - caso Base: UL (e)
 - $\text{fmap f (UL (e))} = \text{UL (fmap f (e))}$ -> Si usa la `fmap` già definita per il tipo `Either a`, quindi:
 - $\text{fmap f (UL (Left x))} = \text{UL (Left x)}$
 - $\text{fmap f (UL (Right y))} = \text{UL (Right f y)}$
 - caso Induttivo: UN (l, r)
 - $\text{fmap f (UN (l, r))} = \text{UN (fmap f l, fmap f r)}$
2. La legge di identità di fmap afferma che $\text{fmap id} = \text{id}$ (quindi non modifica la struttura)
 - caso Base: UL (e)
 - $\text{fmap id (UL (e))} = \text{UL (fmap id (e))} = \text{UL (e)} = \text{id (UL (e))}$
 - caso Induttivo: UN (l, r)
 - $\text{fmap id (UN (l, r))} = \text{UN (fmap id l, fmap id r)} = \text{UN (l, r)} = \text{id (UN (l, r))}$

Ex15:

Consider the following Haskell data-type:

data U a b where

UL:: Either a b -> U a b

UN:: (U a b, U a b) -> U a b

1. Define the function fmap.
2. Prove that fmap enjoys the composition law.

Soluzione:

Stessi discorsi dell'esercizio precedente, ma con la legge della composizione

- 2) La legge della composizione di fmap afferma che $\text{fmap (f . g)} = \text{fmap f . fmap g}$
 - caso Base: UL (e)
 - Sinistra: $\text{fmap (f . g) (UL (e))} = \text{UL (fmap (f . g) e)} = (\text{H.P.}) \text{UL (fmap f . fmap g e)}$

- Destra: $\text{fmap } f . \text{fmap } g (\text{UL } (e)) = \text{fmap } f . (\text{UL } (\text{fmap } g e)) = \text{UL } (\text{fmap } f . \text{fmap } g e)$
- caso Induttivo: $\text{UN } (l, r)$
- Sinistra: $\text{fmap } (f . g) (\text{UN } (l, r)) = \text{UN } (\text{fmap } (f . g) l, \text{fmap } (f . g) r) \stackrel{\text{(H.P.)}}{=} \text{UN } (\text{fmap } f . \text{fmap } g l, \text{fmap } f . \text{fmap } g r) = \text{UN } (\text{fmap } f (\text{fmap } g l), \text{fmap } f (\text{fmap } g r))$
- Destra: $\text{fmap } f . \text{fmap } g (\text{UN } (l, r)) = \text{fmap } f . (\text{UN } (\text{fmap } g l, \text{fmap } g r)) = \text{UN } (\text{fmap } f (\text{fmap } g l), \text{fmap } f (\text{fmap } g r))$

Ex16:

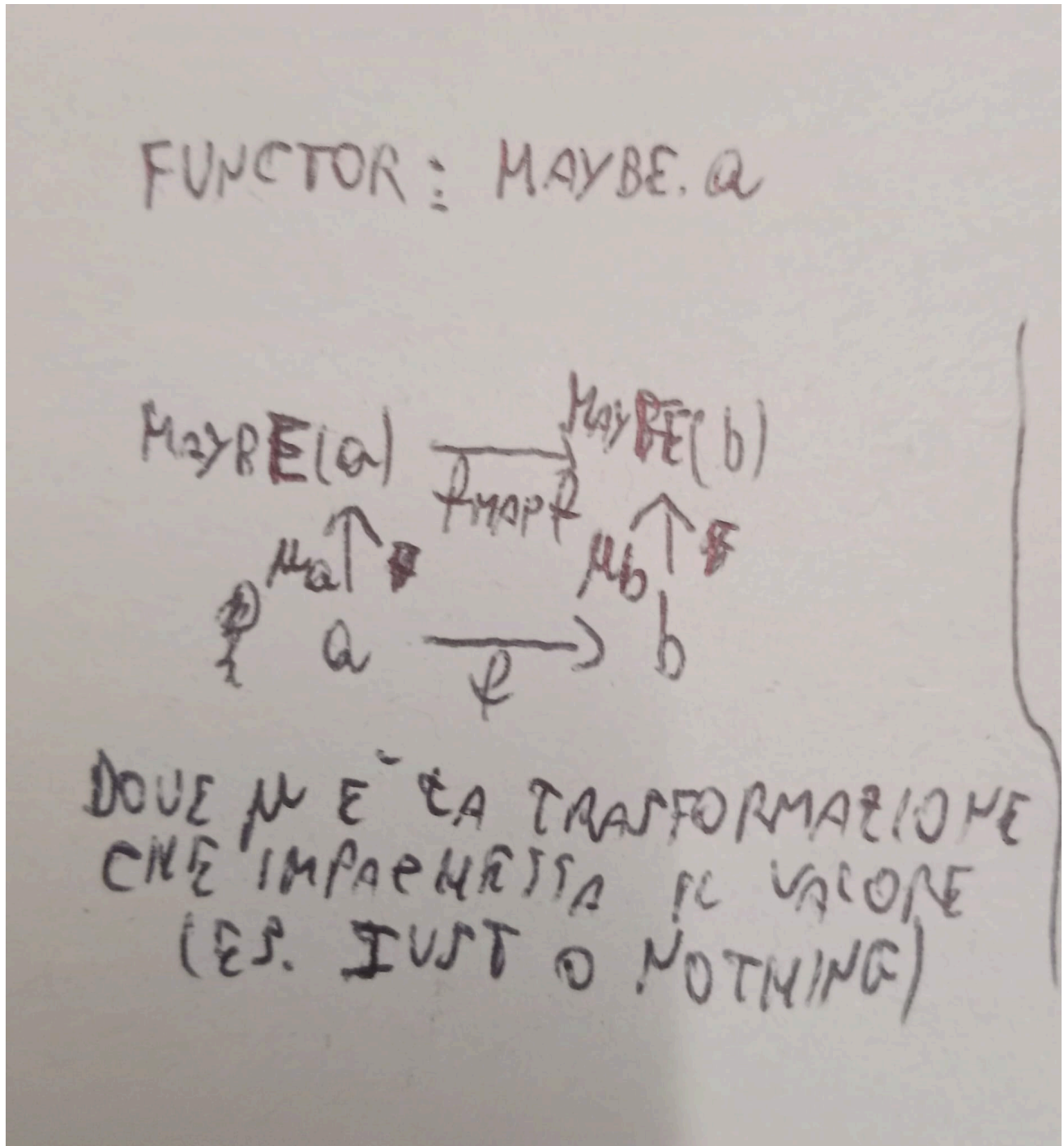
Let us assume that we are in the category Set.

1. Recall the definition of “Maybe a”.
2. Draw the corresponding diagram, starting from an arrow “ $f :: a \rightarrow b$ ”.
3. Decorate the diagram objects with simple instances.
4. Define a simple graph for “ $f :: a \rightarrow b$ ”.
5. Describe how $\text{fmap } f$ would work, without explicitly defining it.

Soluzione:

1. Nella categoria Set, il functor "Maybe" agisce su un insieme A aggiungendo un elemento che rappresenta un valore nullo; data $\text{Maybe } a = \text{Nothing} \mid \text{Just } a$

2. $f :: a \rightarrow b$



3. Esempio di istanze; $A = \{1, 2\}$; $B = \{'c', 'd'\}$; $f(1) = 'c'$; $f(2) = 'd'$
- Oggetti decorati: $Maybe\ A = \{Nothing, Just\ 1, Just\ 2\}$; $Maybe\ B = \{Nothing, Just\ 'c', Just\ 'd'\}$
4. Il grafo di una funzione è l'insieme delle coppie ordinate $(a, f(a))$, quindi $Gf = \{(1, 'c'), (2, 'd')\}$
- Insiemi di nodi $\{1, 2, 'c', 'd'\}$ e di archi diretti $1 \rightarrow 'c'$ e $2 \rightarrow 'd'$
5. $fmap$ agisce su f solo se il valore esiste, preservando la struttura del contenitore;
- quindi se riceve un valore impacchettato (come "Just x") applica la funzione f e restituisce il risultato (es. "Just (f x)")
 - altrimenti se riceve un valore vuoto (come "Nothing") allora ignora e lo restituisce

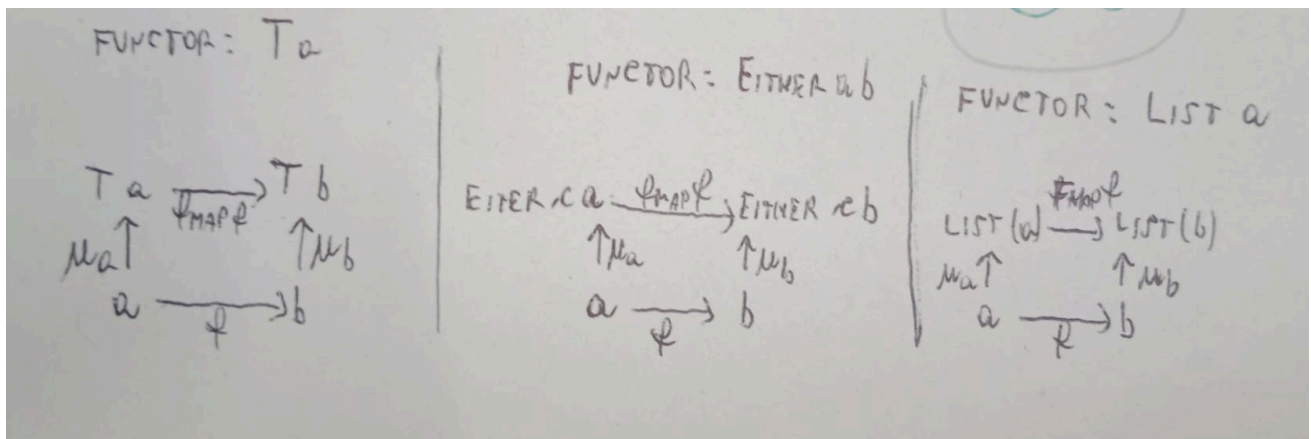
Ex17:

Let us assume that we are in the category Set.

1. Define a data-type "List a" that recursively formalizes the corresponding concept.
2. Draw the corresponding diagram, starting from an arrow " $f :: a \rightarrow b$ ".
3. Decorate the diagram objects with simple instances.
4. Define a simple graph for " $f :: a \rightarrow b$ ".
5. Describe how `fmap f` would work, without explicitly defining it.

Soluzione:

1. data List a = Nil | Const a (List a)
2. $f :: a \rightarrow b$



3. Esempio di istanze; $A = \{1, 2\}$; $B = \{'c', 'd'\}$; $f(1) = 'c'$; $f(2) = 'd'$
 - Oggetti decorati: List A = {Nil, [1], [2], [1, 2], [2, 1], [1, 1, 2], ...}; List B = {Nil, [c], [d], [c, d], [d, c], [c, c, d], ...}
4. Il grafo di una funzione è l'insieme delle coppie ordinate $(a, f(a))$, quindi $Gf = \{(1, 'c'), (2, 'd')\}$
 - Insiemi di nodi $\{1, 2, 'c', 'd'\}$ e di archi diretti $1 \rightarrow 'c'$ e $2 \rightarrow 'd'$
5. `fmap` agisce su `f` solo se il valore esiste, preservando la struttura del contenitore;
 - quindi se riceve la lista come valore composta da A allora si applica la funzione `f` ad ogni singolo elemento e si riceve la lista risultante di B senza invariare la sua struttura

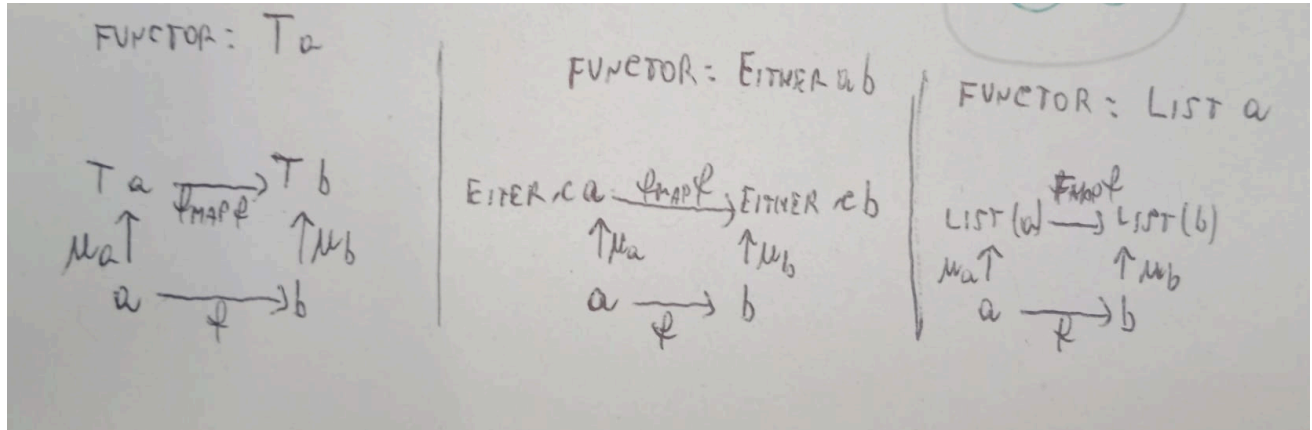
Ex18:

Let us assume that we are in the category Set.

1. Recall the definition of "Either a b".
2. Draw the corresponding diagram, starting from an arrow " $f :: a \rightarrow b$ ".
3. Decorate the diagram objects with simple instances.
4. Define a simple graph for " $f :: a \rightarrow b$ ".
5. Describe how `fmap f` would work, without explicitly defining it.

Soluzione:

1. Nella categoria Set, il functor "Either" rappresenta la somma disgiunta di due insiemi A e B;
data $\text{Either } a \ b = \text{Left } a \mid \text{Right } b$
2. $f :: a \rightarrow b$



3. Esempio di istanze; $A = \{1, 2\}$; $B = \{'c', 'd'\}$; $f(1) = 'c'$; $f(2) = 'd'$; $C = \{\text{True}, \text{False}\}$
 - Oggetti decorati: $\text{Either } C \ A = \{\text{Left True}, \text{Left False}, \text{Right } 1, \text{Right } 2\}$; $\text{Either } C \ B = \{\text{Left True}, \text{Left False}, \text{Right 'c'}, \text{Right 'd'}\}$
4. Il grafo di una funzione è l'insieme delle coppie ordinate $(a, f(a))$, quindi $Gf = \{(1, 'c'), (2, 'd')\}$
 - Insiemi di nodi $\{1, 2, 'c', 'd'\}$ e di archi diretti $1 \rightarrow 'c'$ e $2 \rightarrow 'd'$
5. `fmap` agisce su `f` solo se il valore esiste, preservando la struttura del contenitore;
 - quindi in caso di `Right` applica la funzione `f` e restituisce il risultato `Right (f a)`
 - altrimenti in caso di `Left` allora ignora e lo restituisce

Ex19:

Consider the following Haskell data-type:

```
data T a where
```

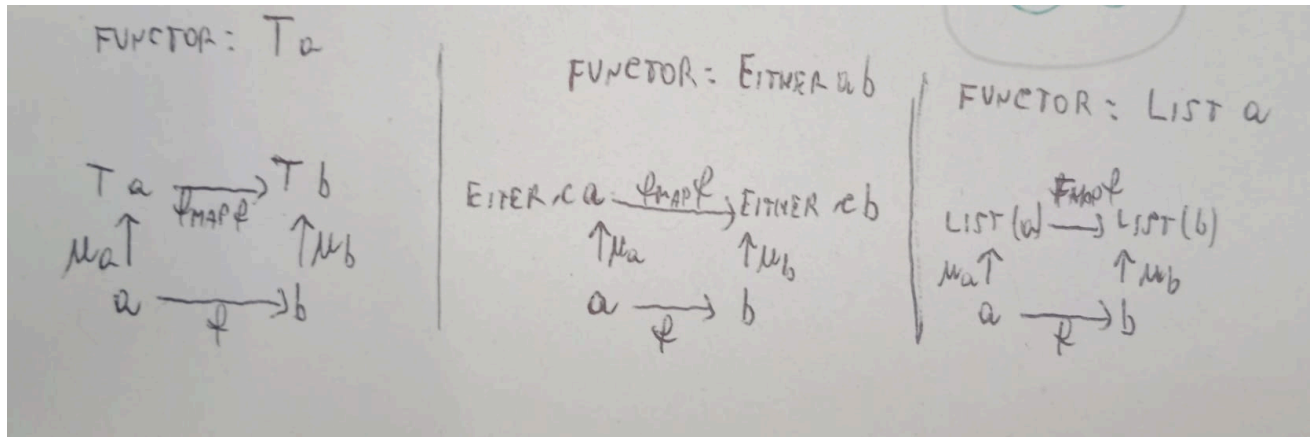
```
TL :: a -> T a
```

```
TN :: [ T a ] -> T a
```

1. Draw the corresponding diagram, starting from an arrow " $f :: a \rightarrow b$ ".
2. Decorate the diagram objects with simple instances.
3. Define a simple graph for " $f :: a \rightarrow b$ ".
4. Describe how `fmap f` would work, without explicitly defining it.

Soluzione:

1. $f :: a \rightarrow b$



2. Esempio di istanze; $A = \{1, 2\}$; $B = \{'c', 'd'\}$; $f(1) = 'c'$; $f(2) = 'd'$

- Oggetti decorati: $T A = \{TL\ 1, TN\ [TL\ 1, TL\ 2], \dots\}$; $Either\ C\ B = \{TL\ 'c', TN\ [TL\ 'c', TL\ 'd'], \dots\}$

3. Il grafo rappresenta una struttura specifica di tipo $T A$ e la sua trasformazione in $T B$ tramite f .

- Un nodo radice con due figli foglia: $TN\ [TL\ 1, TL\ 2] \rightarrow TN\ [TL\ 'c', TL\ 'd']$

4. $fmap$ agisce su f in modo ricorsivo, preservando la struttura del contenitore;

- quindi in caso Base (TL) applica la funzione f e restituisce il risultato $TL\ (f\ a)$
- altrimenti in caso Ricorsivo (TN) mantiene la struttura nodo TN ma applica $fmap\ f$ a ogni elemento di TN

Ex20:

Consider the following Haskell data-type:

data BTree n where

Leaf :: BTree n

Node :: n -> BTree n -> BTree n -> BTree n

- Draw the corresponding diagram, starting from an arrow " $f :: a \rightarrow b$ ".
- Decorate the diagram objects with simple instances.
- Define a simple graph for " $f :: a \rightarrow b$ ".
- Describe how $fmap\ f$ would work, without explicitly defining it.

Soluzione:

1. $f :: a \rightarrow b$

Copia dagli esercizi precedenti ma col il dato BTree n

2. Oggetto a: Int; Oggetto b: String

- Istanza BTree a: Node 5 Leaf Leaf; Istanza BTree b: Node '5' Leaf Leaf

3. Il grafo di una funzione è l'insieme delle coppie ordinate $(a, f(a))$, quindi $Gf = \{(5, '5')\}$

- Insiemi di nodi {5, '5'} e di archi diretti 5 -> '5'
4. fmap agisce su f in modo ricorsivo, preservando la struttura del contenitore;
- quindi in caso Base (Leaf) resta invariato
 - altrimenti in caso Ricorsivo (Node) mantiene la struttura Node e applica fmap f al sottoalbero sx e dx

Ex21:

Consider the following Haskell data-type:

data BTree l where

Leaf :: l -> BTree n l

Node :: BTree n l -> BTree n l -> BTree n l

1. Draw the corresponding diagram, starting from an arrow “f:: a -> b”.
2. Decorate the diagram objects with simple instances.
3. Define a simple graph for “f:: a -> b”.
4. Describe how fmap f would work, without explicitly defining it.

Soluzione:

1. f:: a -> b
Copia dagli esercizi precedenti ma col il dato BTree l
2. Oggetto a: Int; Oggetto b: String
 - Istanza BTree n a: Leaf 5 oppure Node (Leaf 1) (Leaf 2); Istanza BTree n b: Leaf '5' oppure Node (Leaf '1') (Leaf '2')
3. Il grafo di una funzione è l'insieme delle coppie ordinate (a, f(a)), quindi $G_f = \{(5, '5')\}$
 - Insiemi di nodi {5, '5'} e di archi diretti 5 -> '5'
4. fmap agisce su f in modo ricorsivo, preservando la struttura del contenitore;
 - quindi in caso Base (Leaf) applica la funzione (f) al valore contenuto nella foglia
 - altrimenti in caso Ricorsivo (Node) mantiene la struttura Node e applica fmap f al sottoalbero sx e dx

CT0380 Applicative functors

Ex1:

Justify why applicative functors are useful for programmers, using an example.

Soluzione:

Mentre un Functor permette di applicare una funzione normale (che non appartiene a nessun

contesto) ad un valore contenuto in un contesto ($\text{fmap} :: (a \rightarrow b) \rightarrow F a \rightarrow F b$)

funzione normale

↓

fmap

↓

contenitore

Gli Applicative permettono di applicare una funzione contenuta nel contesto ad un valore contenuto nello stesso contesto, quindi combina valori che vivono nello stesso contesto
 $\text{contenitore}(\text{funzione})$

+

$\text{contenitore}(\text{valore})$

↓

$<*>$

↓

$\text{contenitore}(\text{risultato})$

Esempio con "Maybe":

- Functor: $\text{fmap } (+1) (\text{Just } 3) = \text{Just } 4$ (funzione $(+1)$ non appartiene a nessun contesto)
- Applicative: $\text{Just } (+1) <*> \text{Just } 3 = \text{Just } 4$ (funzione $(+1)$ è contenuta nel contesto di "Maybe") oppure $\text{pure } (+) <*> \text{Just } 2 <*> \text{Just } 3 = \text{Just } 4$

N.B.

- $<*>$ si legge "prendi una funzione dentro un contesto e applicala a un valore dentro lo stesso contesto"
 - $(<*>) :: F (a \rightarrow b)$
 $\rightarrow F a$
 $\rightarrow F b$
- pure serve a mettere un valore normale nel contesto (es. $\text{pure } (+) = \text{Just } (+)$)
 - $\text{pure} :: a \rightarrow F a$

CT0390 Applicative functors in Haskell

Ex2:

Recall the definition of the data-type "Maybe a". Assume that you already proved that it is a functor.

1. Define the functions pure and $(<*>)$.
2. Prove that $(<*>)$ enjoys the applicative composition law.

Soluzione:

data Maybe a :: Nothing | Just a

1. Pure:

- `pure :: a -> Maybe a`
- quindi si ha `pure x = Just x`

2. (`<*>`):

- `( <*> ) :: Maybe (a -> b)`
`-> Maybe a`
`-> Maybe b`
- quindi si ha:
 - `Nothing ( <*> ) _ = Nothing`
 - `_ ( <*> ) Nothing = Nothing`
 - `Just f ( <*> ) Just x = Just (f x)`

3. La legge di composizione afferma che: `pure (.) <*> u <*> v <*> w = u <*> (v <*> w)`;
dove :

- `u :: Maybe (b -> c)`, si ha `u = Just f`
- `v :: Maybe (a -> b)`, si ha `v = Just g`
- `w :: Maybe a`, si ha `w = Just x`
- Sinistra:
 - `pure (.) <*> Just f <*> Just g <*> w = Just (.) <*> Just f <*> Just g <*> Just x`
`= Just (.) <*> Just f <*> Just g <*> Just x = Just ((.) f) <*> Just g <*> Just x`
`= Just (f . g) <*> Just x = Just (f . g x) = Just (f (g x))`
- Destra:
 - `Just f <*> (Just g <*> Just x) = Just f <*> (Just (g x)) = Just (f (g x))`

N.B. se compare un `Nothing` anche gli altri producono `Nothing`

Ex3:

Recall the definition of the data-type “Either a b”. Assume that you already proved that it is a functor.

1. Define the functions `pure` and `( <*> )`.
2. Prove that `( <*> )` enjoys the applicative composition law.

Soluzione:

data Either a b :: Left a | Right b

1. Pure:

- $\text{pure} :: b \rightarrow \text{Either } a \ b$
 - quindi si ha $\text{pure } x = \text{Either } a \ x = \text{Right } x$

2. $(\<*\>)$:

- $(\<*\>) :: \text{Either } a \ (b \rightarrow c)$
 - $\rightarrow \text{Either } a \ b$
 - $\rightarrow \text{Either } a \ c$
- quindi si ha:
 - $\text{Left } a \ (\<*\>) _ = \text{Left } a$
 - $_ \ (\<*\>) \text{Left } a = \text{Left } a$
 - $\text{Right } f \ (\<*\>) \text{Right } x = \text{Right } (f \ x)$

3. La legge di composizione afferma che: $\text{pure } (.) \ \<*\> \ u \ \<*\> \ v \ \<*\> \ w = u \ \<*\> \ (v \ \<*\> \ w)$; dove :

- $u :: \text{Either } e \ (b \rightarrow c)$, si ha $u = \text{Right } f$
- $v :: \text{Maybe } e \ (a \rightarrow b)$, si ha $v = \text{Right } g$
- $w :: \text{Maybe } e \ a$, si ha $w = \text{Right } x$
- Sinistra:
 - $\text{pure } (.) \ \<*\> \ \text{Right } f \ \<*\> \ \text{Right } g \ \<*\> \ \text{Right } x = \text{Right } (.) \ \<*\> \ \text{Right } f \ \<*\> \ \text{Right } g \ \<*\> \ \text{Right } x = \text{Right } ((.) \ f) \ \<*\> \ \text{Right } g \ \<*\> \ \text{Right } x = \text{Right } (f. \ g) \ \<*\> \ \text{Right } x = \text{Right } (f. \ g \ x) = \text{Right } (f \ (g \ x))$
- Destra:
 - $\text{Right } f \ \<*\> \ (\text{Right } g \ \<*\> \ \text{Right } x) = \text{Right } f \ \<*\> \ (\text{Right } (g \ x)) = \text{Right } (f \ (g \ x))$

N.B. se compare un Left anche gli altri producono Left

Ex4:

Define a data-type “List a” that recursively formalizes the corresponding concept.

1. Show that it can be a functor by defining the function `fmap`.
2. Define the functions `pure` and `(< * >)`.
3. Recall the corresponding applicative composition law, explicitly writing the types its components.

Soluzione:

`data List a :: Nil | Cons a (List a)`

1. Si applica List come Functor tramite `fmap`; $\text{fmap} :: (a \rightarrow b) \rightarrow \text{List } a \rightarrow \text{List } b$
 - $\text{fmap } f \ \text{Nil} = \text{Nil}$
 - $\text{fmap } f \ (\text{Cons } x \ xs) = \text{Cons } (f \ x) \ (\text{fmap } f \ xs)$
2. Pure:

- `pure :: a -> List a`
 - quindi si ha `pure x = List x = Cons x Nil` (lista singleton)

3. `(<*>)`:

- `(<*>) :: List (a -> b)`
`-> List a`
`-> List b`
- quindi si ha:
 - `Nil (<*>) _ = Nil`
 - `_ (<*>) Nil = Nil`
 - `Cons f fs (<*>) xs = append (fmap f xs) (fs (<*>) xs)`
 - N.B. per implementarla ricorsivamente si usa la funzione `append` per concatenare le due liste
 - Si prende la prima funzione `f` dalla lista di funzioni
 - La si mappa nell'intera lista di valori `xs` (usando `fmap`)
 - Ricorsivamente lo si fa per le altre funzioni `fs`
 - E infine si uniscono le liste risultanti insieme
 - Es. `[(+1), (*2)] (<*>) [1, 2] = [2, 3, 2, 4]`

4. La legge di composizione afferma che: `pure (.) (<*>) u (<*>) v (<*>) w = u (<*>) (v (<*>) w)`;
 dove :

- `(.) :: (b->c) -> (a->b) -> a -> c`
- `pure (.) :: List ( (b->c) -> (a->b) -> a -> c )`
- `u :: List (b -> c)`
- `v :: List (a -> b)`
- `w :: List a`
- quindi sia `sx` che `dx` generano `g (f x)` con `g ∈ u`, `f ∈ v`, `x ∈ w`
- N.B. L'esercizio richiede di richiamare la legge (esplicitando i tipi dei componenti) e non di provarla

Ex5:

Consider the following Haskell data-type:

`newtype Reader r a where`

`Reader :: (r -> a) -> Reader r a`

1. Show that it can be a functor by defining the function `fmap`.
2. Define the functions `pure` and `(<*>)`.
3. Recall the corresponding applicative composition law, explicitly writing the types its components.

Soluzione:

`newtype Reader r a = Reader (r -> a)`

E' essenzialmente un contenitore per una funzione che aspetta un input di tipo `r` (l'ambiente) per restituire un risultato di tipo `a`

Scrivere `Reader (\r -> ...)` si dice "Non eseguire questo codice ora; aspetta che qualcuno mi passi un valore `r`"

1. Si applica `Reader` come Functor tramite `fmap`; `fmap :: (a -> b) -> Reader r a -> Reader r b`

- `fmap f (Reader g) = Reader (f . g) = Reader (\r -> f (g r))`
- `Reader` si aspetta una funzione come argomento, quindi si usa il lambda calcolo per definire come calcolare il nuovo risultato quando si riceve `r`: prima lo passiamo a `g` e poi il risultato a `f` (opzionale da definire)

2. `Pure`:

- `pure :: a -> Reader r a`
 - `pure x = Reader (\r -> x)`
 - crea un `Reader` che ignora l'input (l'ambiente '`r`') e restituisce sempre lo stesso valore '`x`' (funzione costante)

3. `(<*>)`:

- `(<*>) :: Reader r (a -> b) -> Reader r a -> Reader r b`
- quindi si ha:
 - `Reader f (<*>) Reader g = Reader (\r -> f r (g r))`
 - `f r`: Passiamo l'ambiente `r` alla prima funzione (`f = r -> (a -> b)`) per ottenere la funzione "reale" di tipo `(a->b)`
 - `g r`: Passiamo lo stesso ambiente `r` alla seconda funzione (`g = r -> a`) per ottenere il valore di tipo `a`.
 - `f r (g r)`: Applichiamo la funzione al valore per ottenere `b`

4. La legge di composizione afferma che: `pure (.) <*> u <*> v <*> w = u <*> (v <*> w)`; dove :

- `(.) :: (b->c) -> (a->b) -> a -> c`
- `pure (.) :: Reader r ( (b->c) -> (a->b) -> a -> c )`
- `u :: Reader r (b -> c)`
- `v :: Reader r (a -> b)`
- `w :: Reader r a`
- quindi sia `sx` che `dx` generano `(g r) ((f r) (x r))` con `g ∈ u`, `f ∈ v`, `x ∈ w`
- N.B. L'esercizio richiede di richiamare la legge (esplicitando i tipi dei componenti) e non di provarla

Ex6:

Consider the following Haskell data-type:

```
newtype Diagonal a = Diagonal {runDiagonal :: (,) a a}
```

1. Show that it can be a functor by defining the function `fmap`.
2. Define the functions `pure` and `( <*> )`.
3. Recall the corresponding applicative composition, explicitly writing the types of the categorical objects involved.

Soluzione:

```
newtype Diagonal a = Diagonal (a, a)
```

1. Si applica `Diagonal` come `Functor` tramite `fmap`; `fmap :: (a -> b) -> Diagonal (a, a) -> Diagonal (b, b)`
 - `fmap f (Diagonal (x, y)) = Diagonal (f x, f y)`
2. `Pure`:
 - `pure :: a -> Diagonal (a, a)`
 - `pure x = Diagonal (x, x)`
 - replica il valore in entrambe le posizioni
3. `( <*> )`:
 - `( <*> ) :: Diagonal (a -> b, a -> b) -> Diagonal (a, a) -> Diagonal (b, b)`
 - quindi si ha:
 - `Diagonal (f, g) ( <*> ) Diagonal (x, y) = Diagonal (f x, g y)`
4. La composizione agisce su $(F.G)(a) = \text{Diagonal}(\text{Diagonal } a)$; i tipi sono:
 - `u :: Diagonal (b -> c)`; quindi se `u = (g1, g2)`
 - `v :: Diagonal (a -> b)`; quindi se `v = (f1, f2)`
 - `w :: Diagonal a`; quindi se `w = (x1, x2)`
 - si ha che sia `sx` che `dx` generano `(g1 (f1 x1), g2 (f2 x2))` con `g ∈ u`, `f ∈ v`, `x ∈ w`
 - N.B. L'esercizio richiede di richiamare la legge (esplicitando i tipi dei componenti) e non di provarla

Ex7:

Recall the definition of the Haskell data-type:

```
newtype Const c a = Const { getConst :: c }
```

Assume that you already proved that it is a functor (and assume that `c` is a monoid).

1. Define the functions `pure` and `( <*> )`.

2. Prove that (<*>) enjoys the applicative composition law.

Soluzione:

`newtype Const c a = Const c`

Il parametro 'a' è un phantom type, definito perchè:

- Compatibilità con i Functor/Applicative: Per definire un'istanza di Functor o Applicative, il tipo deve avere "kind" `* -> *` (ovvero deve accettare un parametro di tipo). Anche se `Const` memorizza internamente solo un valore di tipo `c`, l'aggiunta di `a` permette a `Const c` di comportarsi come un contenitore che "sembra" contenere un valore di tipo `a`.
- Flessibilità nel `fmap`: Quando usi `fmap`, la funzione trasforma un `f a` in un `f b`. Nel caso di `Const`, l'operazione di mappatura cambia il "tag" del tipo da `a` a `b` senza modificare il valore reale contenuto (`c`), permettendo al codice di compilare correttamente all'interno di pipeline di funzioni generiche.

In sintesi, `a` serve a soddisfare i requisiti del sistema dei tipi per permettere a `Const c` di essere usato dove è richiesto un funtore, ignorando però il valore "fittizio" durante il calcolo effettivo.

1. Pure:

- `pure :: a -> Const c`
 - `pure :: _ -> Const mempty`
 - dove `mempty` è l'elemento neutro di un monoide
 - ignora l'argomento `a` e usa l'identità del monoide per `c`

2. (<*>):

- `( <*> ) :: Const c (a -> b)`
`-> Const c a`
`-> Const c b`
- quindi si ha:
 - `Const x ( <*> ) Const y = Const (x <> y)`
 - dove `<>` è l'operazione del monoide

3. La legge di composizione afferma che: `pure (.) <*> u <*> v <*> w = u <*> (v <*> w)`;
dove :

- `u :: Const c (b -> c)`, si ha `u = Const x`
- `v :: Const c (a -> b)`, si ha `v = Const y`
- `w :: Const c a`, si ha `w = Const z`
- Sinistra:
 - `pure (.) <*> Const x <*> Const y <*> Const z = Const mempty <*> Const x <*> Const y <*> Const z = Const ((mempty <> x) <> y <> z) = Const ((x <> y) <> z)`
- Destra:

- $\text{Const } x \text{ <*> } (\text{Const } x \text{ <*> } \text{Const } z) = \text{Const } (x \text{ <> } (y \text{ <> } z))$
- Proprietà associativa $(x \text{ <> } y) \text{ <> } z = x \text{ <> } (y \text{ <> } z)$

Ex8:

Recall the definition of the Haskell data-type:

```
newtype ZipList a = ZipList { getZipList :: [ a ] }
```

Assume that you already proved that it is a functor by defining:

```
instance Functor ZipList where
```

```
fmap :: (a -> b) -> ZipList a -> ZipList b
```

```
fmap f zl = ZipList (map f (getZipList zl))
```

1. Define the functions `pure` and `( <*> )` (the latter acting pointwise).
2. Prove that `( <*> )` enjoys the applicative composition law.

Soluzione:

```
newtype ZipList a = ZipList [ a ]
```

`ZipList` serve a modificare il comportamento di default delle liste quando vengono usate come `Applicative Functor`. Invece di combinare tutti gli elementi in modo incrociato (prodotto cartesiano) (default),

applica le funzioni agli elementi con lo stesso indice in contemporanea.

- **List standard** `[]` : Applicando una funzione con più argomenti a più liste, ottieni tutte le combinazioni possibili.
`(+) <$> [1, 2] <*> [10, 20]`
Risultato: `[11, 21, 12, 22]`
- **ZipList** : Esegue l'operazione posizionando gli elementi in parallelo (il primo con il primo, il secondo con il secondo) e si ferma alla lista più corta.
`(+) <$> ZipList [1, 2] <*> ZipList [10, 20]`
Risultato: `ZipList {getZipList = [11, 22]}`

1. Pure:

- `pure :: a -> ZipList [ a ]`
 - `pure :: _ -> ZipList (repeat x)`
 - crea una lista infinita dello stesso elemento

2. `( <*> )`:

- `( <*> ) :: ZipList (a -> b)`
`-> ZipList a`
`-> ZipList b`
- quindi si ha:
 - `ZipList fs ( <*> ) ZipList xs = ZipList (zipWith ($) fs xs)`

- dove `zipWith` è una funzione che prende due liste e le combina usando una funzione passata come primo argomento
- l'operatore '\$' applica la funzione a sx al valore a dx
- l'espressione equivale a fare: `zipWith (\f x -> f x)`
- esempio:
 - `ZipList [(+1),(*2)] <*> ZipList [10,20] = ZipList [11,40]`

3. La legge di composizione afferma che: `pure (.) <*> u <*> v <*> w = u <*> (v <*> w)`;
dove :

- `u :: ZipList fs`, si ha `u = [f1, f2, ...]`
- `v :: ZipList gs`, si ha `v = [g1, g2, ...]`
- `w :: ZipList xs`, si ha `w = [x1, x2, ...]`
- Sinistra:
 - `pure (.) <*> ZipList fs <*> ZipList gs <*> ZipList xs = ZipList [f1 . , f2 . , ...] <*> ZipList gs <*> ZipList xs = ZipList [f1 . g1, f2 . g2, ...] <*> ZipList xs = ZipList [f1 . g1 (x1), f2 . g2 (x2), ...]`
- Destra:
 - `u <*> (v <*> w) = ZipList fs <*> (ZipList gs <*> ZipList xs) = ZipList fs <*> (ZipList [g1 (x1), g2 (x2), ...]) = ZipList [f1 (g1 (x1)), f2 (g2 (x2)), ...]`

Ex9:

Recall the definition of the Haskell data-type:

```
newtype Identity a = Identity { runIdentity :: a }
```

Assume that you already proved that it is a functor by defining:

1. Define the functions `pure` and `( <*> )`.
2. Prove that `( <*> )` enjoys the applicative composition law.

Soluzione:

```
newtype Identity a = Identity a
```

Il functor `Identity` della classe `Applicative` è utilizzato per avvolgere un valore senza modificarne la natura. Viene spesso impiegato per soddisfare i requisiti del compilatore, ad esempio quando devi passare un valore semplice a una funzione che richiede un contesto di tipo `Monad` o `Functor`

1. `Pure`:
 - `pure :: a -> Identity a`
 - `pure :: x -> Identity x`
2. `( <*> )`:

- $(\langle * \rangle) :: \text{Identity } (a \rightarrow b)$
 $\rightarrow \text{Identity } a$
 $\rightarrow \text{Identity } b$
- quindi si ha:
 - $\text{Identity } f (\langle * \rangle) \text{ Identity } x = \text{Identity } (f x)$

3. La legge di composizione afferma che: $\text{pure } (.) \langle * \rangle u \langle * \rangle v \langle * \rangle w = u \langle * \rangle (v \langle * \rangle w)$;

dove :

- $u :: \text{Identity } f$
- $v :: \text{Identity } g$
- $w :: \text{Identity } x$
- Sinistra:
 - $\text{pure } (.) \langle * \rangle \text{ Identity } f \langle * \rangle \text{ Identity } g \langle * \rangle \text{ Identity } x = \text{Identity } (.) \langle * \rangle \text{ Identity } f \langle * \rangle \text{ Identity } g \langle * \rangle \text{ Identity } x = \text{Identity } ((.) f) \langle * \rangle \text{ Identity } g \langle * \rangle \text{ Identity } x = \text{Identity } (f . g) \langle * \rangle \text{ Identity } x = \text{Identity } (f . g (x))$
- Destra:
 - $u \langle * \rangle (v \langle * \rangle w) = \text{Identity } f \langle * \rangle (\text{Identity } g \langle * \rangle \text{ Identity } x) = \text{Identity } f \langle * \rangle (\text{Identity } (g x)) = \text{Identity } (f (g x))$

CT0450 Natural transformations in Haskell

Una natural transformation è una trasformazione tra due functor (F e G) che preserva la struttura: famiglia di funzioni: $\eta :: F a \rightarrow G a$; valida per ogni tipo di a

L'idea è trasformare un contenitore in un altro senza guardare il contenuto

```
maybeToList :: Maybe a -> '[a]`
maybeToList Nothing  = '[]`
maybeToList (Just x) = '[x]`
```

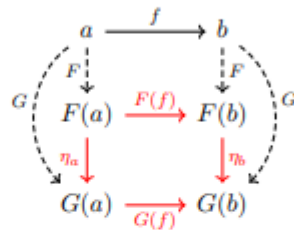
Natural Transformation

Assumption.

$\mathcal{C} \xrightarrow{F} \mathcal{D}$ functors with the same domain and codomain.

Natural transformation from F to G .

Family $\{\eta_a : F \rightarrow G\}_{a \in \mathcal{C}}$ s.t. the red "box":



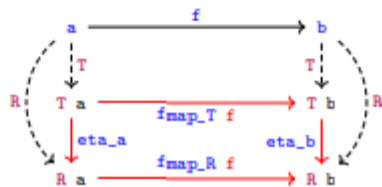
commutes for every morphism $f : a \rightarrow b$ in $\mathcal{C}$, i.e.:

$$\eta_b \circ F(f) = G(f) \circ \eta_a$$

eta :: Term a -> Result a is natural

Let rename **Term** as **T** and **Result** as **R**.

For every $f :: a \rightarrow b$, the diagram:



commutes.

Ex1:

Recall the definition of natural transformation between two functors F and G

Soluzione:

Una natural transformation tra due functor: $F, G :: \rightarrow$

è una famiglia di funzioni $\eta :: F a \rightarrow G a$

tale che per ogni funzione $f :: a \rightarrow b$

vale:

$$\eta \cdot \text{fmap}_F f = \text{fmap}_G f \cdot \eta$$

Ex2:

Let the following two Haskell functors `Maybe` and `[]` be given.

Consider the following Haskell function

`maybeToList :: Maybe a -> [a]`

`maybeToList Nothing = []`

`maybeToList (Just x) = [x]`

Prove the naturality law in the case where the argument is `Just x`.

Soluzione:

Si prova la legge della naturalità nel caso di `Just x`

Si dimostra che `maybeToList . fmap f (Just x) = fmap f . maybeToList (Just x)`

- Sinistra:
 - `maybeToList . fmap f (Just x) = maybeToList . Just (f x) = maybeToList (Just (f x)) = [f x]`
- Destra:
 - `fmap f . maybeToList (Just x) = fmap f . [x] = fmap f ([x]) = [f x]`

Ex3:

Let the following two Haskell functors `[]` and `Maybe` be given.

Consider the following Haskell function

`listToMaybe :: [a] -> Maybe a`

`listToMaybe [] = Nothing`

`listToMaybe (x: _) = Just x`

Prove the naturality law in the case where the argument is `h:t`.

Soluzione:

Si prova la legge della naturalità nel caso di `h:t`

Si dimostra che `listToMaybe . fmap f (h:t) = fmap f . listToMaybe (h:t)`

- Sinistra
 - `listToMaybe . fmap f (h:t) = listToMaybe (f h : fmap f t) = Just (f h)`
- Destra
 - `fmap f . listToMaybe (h:t) = fmap f (Just h) = Just (f h)`

Ex4:

Let the following two Haskell functors `Maybe` and `Either ()` be given.

Consider the following Haskell function

`toEitherUnit :: Maybe a -> Either () a`

`toEitherUnit Nothing = Left ()`

`toEitherUnit (Just x) = Right x`

Prove the naturality law in the case where the argument is `Just x`.

Soluzione:

Si prova la legge della naturalità nel caso di `Just x`

Si dimostra che `toEitherUnit . fmap f (Just x) = fmap f . toEitherUnit (Just x)`

- Sinistra
 - `toEitherUnit . fmap f (Just x) = toEitherUnit (Just (f x)) = Right (f x)`

- Destra
 - $\text{fmap } f . \text{toEitherUnit } (\text{Just } x) = \text{fmap } f (\text{Right } x) = \text{Right } (f \ x)$

Ex5:

Let the following two Haskell functors Maybe and Either () be given.

Consider the following Haskell function

$\text{toEitherUnit} :: \text{Maybe } a \rightarrow \text{Either } () \ a$

$\text{toEitherUnit } \text{Nothing} = \text{Left } ()$

$\text{toEitherUnit } (\text{Just } x) = \text{Right } x$

Prove the naturality law in the case where the argument is Nothing.

Soluzione:

Si prova la legge della naturalità nel caso di Nothing

Si dimostra che $\text{toEitherUnit} . \text{fmap } f (\text{Nothing}) = \text{fmap } f . \text{toEitherUnit } (\text{Nothing})$

- Sinistra
 - $\text{toEitherUnit} . \text{fmap } f (\text{Nothing}) = \text{toEitherUnit } (\text{Nothing}) = \text{Left } ()$
- Destra
 - $\text{fmap } f . \text{toEitherUnit } (\text{Nothing}) = \text{fmap } f (\text{Left } ()) = \text{fmap } f (\text{Left } ()) = \text{Left } ()$

Ex6:

Let the following two Haskell functors Either () and Maybe be given.

Consider the following Haskell function

$\text{fromEitherUnit} :: \text{Either } () \ a \rightarrow \text{Maybe } a$

$\text{fromEitherUnit } (\text{Left } ()) = \text{Nothing}$

$\text{fromEitherUnit } (\text{Right } x) = \text{Just } x$

Prove the naturality law in the case where the argument is Right x.

Soluzione:

Si prova la legge della naturalità nel caso di Right x

Si dimostra che $\text{fromEitherUnit} . \text{fmap } f (\text{Right } x) = \text{fmap } f . \text{fromEitherUnit } (\text{Right } x)$

- Sinistra
 - $\text{fromEitherUnit} . \text{fmap } f (\text{Right } x) = \text{fromEitherUnit } (\text{Right } (f \ x)) = \text{Just } (f \ x)$
- Destra
 - $\text{fmap } f . \text{fromEitherUnit } (\text{Right } x) = \text{fmap } f (\text{Just } x) = \text{Just } (f \ x)$

Ex7:

Let the following two Haskell functors [] and Either () be given.

Consider the following Haskell function

`headEither :: [a] -> Either () a`

`headEither [] = Left ()`

`headEither (x:_) = Right x`

Prove the naturality law in the case where the argument is `h:t`.

Soluzione:

Si prova la legge della naturalità nel caso di `h:t`

Si dimostra che `headEither . fmap f (h:t) = fmap f . headEither (h:t)`

- Sinistra
 - `headEither . fmap f (h:t) = headEither (f h : fmap (f t)) = Right (f h)`
- Destra
 - `fmap f . headEither (h:t) = fmap f (Right h) = Right (f h)`

Ex8:

Let the following two Haskell functors `Either ()` and `[]` be given.

Consider the following Haskell function

`eitherToList :: Either () a -> [a]`

`eitherToList (Left ()) = []`

`eitherToList (Right x) = [x]`

Prove the naturality law in the case where the argument is `Right x`.

Soluzione:

Si prova la legge della naturalità nel caso di `Right x`

Si dimostra che `eitherToList . fmap f (Right x) = fmap f . eitherToList (Right x)`

- Sinistra
 - `eitherToList . fmap f (Right x) = eitherToList (Right (f x)) = [f x]`
- Destra
 - `fmap f . eitherToList (Right x) = fmap f ([x]) = [f x]`

CT0530 Monad in Haskell

I Monadi estendono gli Applicative e permettono di concatenare computazioni che producono a loro volta valori nel contesto.

L'operazione fondamentale (detta bind):

`(>>=) :: m a`

`-> (a -> m b)`

`-> (m b)`

- si legge: "prendi un valore nel contesto e passalo ad una funzione che restituisce un valore nello stesso contesto"
- tra l'altro m si può vedere come Functor (F)

Esempio:

Supponiamo di avere:

$f :: \text{Int} \rightarrow \text{Maybe Int}$

Ad esempio:

$f\ x = \text{Just}\ (x+1)$

Si parte da:

$\text{Just}\ 3$

Con fmap si ottiene:

$\text{fmap}\ f\ (\text{Just}\ 3) = \text{Just}\ (\text{Just}\ 4)$

Si ottiene un doppio contenitore (si annidano operazioni), per risolvere il problema si concatena:

$\text{Just}\ 3 \gg= f = f\ (3)$

risultato:

$\text{Just}\ 4$

altro esempio con Nothing:

$\text{Nothing} \gg= _$

risultato:

Nothing

N.B. si possono comporre le computazioni ($g :: \text{Int} \rightarrow \text{Maybe Int}$)

- $\text{Just}\ 3 \gg= f \gg= g$

$f\ 3 = \text{Just}\ 4$

$g\ 4 = \text{Just}\ 8$

- risultato: $\text{Just}\ 8$

- Con valori nulli come Nothing si propaga nella composizione

e compare anche il return che permette di prendere un valore e inserirlo all'interno di una monade (di un contenitore)

$\text{return} :: a \rightarrow m\ a$

per Maybe:

$\text{return}\ x = \text{Just}\ x$

inoltre si può provare con la legge di associatività per $\gg=$ (le monadi)

$(m \gg= f) \gg= g = m \gg= (\backslash x \rightarrow f(x)) \gg= g$

Ex9:

Consider the applicative functor Maybe a.

1. Define the function ($\gg=$) for the functor Maybe a.
2. Prove the associativity law for ($\gg=$).

Soluzione:

1. Definizione di bind per il functor Maybe a:

$(\gg=) :: \text{Maybe } a \rightarrow (a \rightarrow \text{Maybe } b) \rightarrow \text{Maybe } b$

- $\text{Nothing } \gg= f$
 $= \text{Nothing}$
 $(\gg=) \text{ Nothing } f = \text{Nothing}$
- $\text{Just } x \gg= f$
 $= f \ x$
 $(\gg=) (\text{Just } x) f = f \ x$
 $\text{return} :: a \rightarrow \text{Maybe } a$
- return pure

2. La legge dell'associatività per $\gg=$ afferma che $(m \gg= f) \gg= g = m \gg= (\lambda x \rightarrow f(x)) \gg= g$

- con il dato m (Maybe) si ha per due casi:
- caso $m = \text{Nothing}$
 - Sinistra
 - $(\text{Nothing } \gg= f) \gg= g = \text{Nothing } \gg= g = \text{Nothing}$
 - Destra
 - $\text{Nothing } \gg= (\lambda x \rightarrow f(x)) \gg= g = \text{Nothing}$
- caso $m = \text{Just } x$
 - Sinistra
 - $(\text{Just } x \gg= f) \gg= g = f \ x \gg= g$
 - Destra
 - $\text{Just } x \gg= ((\lambda y \rightarrow f(y)) \gg= g) = ((\lambda y \rightarrow f(y)) \gg= g) \ x = f(x) \gg= g$

Ex10:

Consider the applicative functor Either u.

1. Define the function ($\gg=$) for the functor Either u.
2. Prove the associativity law for ($\gg=$).

Soluzione:

1. Definizione di bind per il functor Either u:

$(\gg=) :: \text{Either } u \ a \rightarrow (a \rightarrow \text{Either } u \ b) \rightarrow \text{Either } u \ b$

- $\text{Left } l \gg= _$
 $= \text{Left } l$

$(\gg=) (\text{Left } l) f = \text{Left } l$

- $\text{Right } r \gg= f$
 $= f r$

$(\gg=) (\text{Right } r) f = f r$

$\text{return} :: a \rightarrow \text{Either } u a$

- return pure

2. La legge dell'associatività per $\gg=$ afferma che $(m \gg= f) \gg= g = m \gg= (\lambda x \rightarrow f(x)) \gg= g$

- con il dato m (Either) si ha per due casi:
- caso $m = \text{Left } l$
 - Sinistra
 - $(\text{Left } l \gg= f) \gg= g = \text{Left } l \gg= g = \text{Left } l$
 - Destra
 - $\text{Left } l \gg= (\lambda x \rightarrow f(x)) \gg= g = \text{Left } l$
- caso $m = \text{Right } r$
 - Sinistra
 - $(\text{Right } r \gg= f) \gg= g = f(r) \gg= g$
 - Destra
 - $\text{Right } r \gg= ((\lambda x \rightarrow f(x)) \gg= g) = ((\lambda x \rightarrow f(x)) \gg= g)(r) = f(r) \gg= g$

Ex11:

Let us assume that we have defined the type:

$\text{data List } a \text{ where}$

$\text{Nil} :: \text{List } a$

$\text{Cons} :: a \rightarrow \text{List } a \rightarrow \text{List } a$

and that we already proved that it is an applicative functor.

1. Define the function $(\gg=)$ for the functor $\text{List } a$.
2. Prove the associativity law for $(\gg=)$.

Soluzione:

$\text{data List } a :: \text{Nil} \mid \text{Cons } a (\text{List } a)$

La firma è data da:

1. Definizione di bind per il functor $\text{List } a$; in cui si applica una funzione che restituisce una nuova lista a ogni elemento e infine concatena i risultati (perché in questo modo si "appiattisce" le liste in un'unica List , essendo che per definizione del bind restituisce una List):
 - quindi prima si definisce una funzione ausiliaria per concatenare le liste 'append':
 - $\text{append} :: \text{List } a \rightarrow \text{List } a \rightarrow \text{List } a$

- $\text{append Nil } ys = ys$
- $\text{append (Cons } x \text{ } xs) \text{ } ys = \text{Cons } x \text{ (append } xs \text{ } ys)$
- ora si applica la definizione di $>>=$
 $(>>=) :: \text{List } a \rightarrow (a \rightarrow \text{List } b) \rightarrow \text{List } b$
 - $\text{Nil } >>= _ = \text{Nil}$
 $(>>=) \text{ Nil } f = \text{Nil}$
 - $\text{Cons } x \text{ } xs >>= f = \text{append (f } x) (xs >>= f)$
 $(>>=) \text{ Cons } x \text{ } xs \text{ } f = \text{append (f } x) ((>>=) xs \text{ } f)$
 $\text{return} :: a \rightarrow \text{List } a$
- return pure

2. La legge dell'associatività per $>>=$ afferma che $(m >>= f) >>= g = m >>= (\lambda x \rightarrow f(x)) >>= g$

- con il dato m (List) si ha per due casi:
- caso $m = \text{Nil}$
 - Sinistra
 - $(\text{Nil } >>= f) >>= g = \text{Nil } >>= g = \text{Nil}$
 - Destra
 - $\text{Nil } >>= (\lambda x \rightarrow f(x)) >>= g = \text{Nil}$
- caso $m = \text{Cons } x \text{ } xs$
 - Sinistra
 - $(\text{Cons } x \text{ } xs >>= f) >>= g = (\text{append (f } x) (xs >>= f)) >>= g$ (proprietà distributiva di append)
 $\text{append (f } x >>= g) ((xs >>= f) >>= g) = (\text{H.P.})$
 $\text{append (f } x >>= g) (xs >>= (\lambda x \rightarrow f(x)) >>= g)$
 - Destra
 - $\text{Cons } x \text{ } xs >>= (\lambda y \rightarrow f(y)) >>= g =$
 $\text{append } (\lambda y \rightarrow f(y) >>= g) \text{ } x) (xs >>= (\lambda y \rightarrow f(y) >>= g)) =$
 $\text{append (f } x >>= g) (xs >>= (\lambda y \rightarrow f(y) >>= g))$
- proprietà distributiva di append :
 - $(\text{append } as \text{ } bs) >>= g == \text{append (as } >>= g) (bs >>= g)$

Ex12:

Let us assume that we have defined the type:

`newtype Diagonal a = PutD {getD :: (,) a a}`

and that we already proved that it is an applicative functor.

1. Define the function $(>>=)$ for `Diagonal a`.

2. Prove the associativity law for ($\gg=$).

Soluzione:

`newtype Diagonal a :: PutD (a, a)`

Per definire l'istanza di Monad, e in particolare l'operatore bind ($\gg=$), dobbiamo estrarre i valori e applicare la funzione di continuazione

Il bind applica la funzione f all'elemento corrispondente alla posizione originale per mantenere la struttura

1. Definizione di bind per il functor Diagonal a:

$(\gg=) :: \text{Diagonal } a \rightarrow (a \rightarrow \text{Diagonal } b) \rightarrow \text{Diagonal } b$

- $\text{PutD } (x, y) \gg= f$
= $\text{PutD } ((\text{fst} . \text{getD} . f) x, (\text{snd} . \text{getD} . f) y)$
= $\text{PutD } ((\text{fst} . \text{getD} . (\lambda z \rightarrow f z)) x, (\text{snd} . \text{getD} . (\lambda z \rightarrow f z)) y)$
 $(\gg=) (\text{PutD } (l, r)) f = \text{PutD } ((\text{fst} . \text{getD} . f) l, (\text{snd} . \text{getD} . f) r)$
- dove fst è il primo elemento del risultato ottenuto da x (quando si fa $f(x)$ si ottiene due nuovi oggetti Diagonal, ma a noi interessa il primo)
- stessa cosa per snd ma si ottiene il secondo risultato

2. La legge dell'associatività per $\gg=$ afferma che $(m \gg= f) \gg= g = m \gg= (\lambda x \rightarrow f(x) \gg= g)$

- con il dato $m (\text{PutD } (x, y))$ si ha:
- Sinistra
 - $(\text{PutD } (x, y) \gg= f) \gg= g =$
 $(\text{PutD } ((\text{fst} . \text{getD} . f) x, (\text{snd} . \text{getD} . f) y)) \gg= g$
 $\text{PutD } ((\text{fst} . \text{getD} . (\lambda z \rightarrow (f z \gg= g))) x, (\text{snd} . \text{getD} . (\lambda z \rightarrow (f z \gg= g))) y) =$
 $\text{PutD } (\text{fst} . \text{getD} . (f x \gg= g), \text{snd} . \text{getD} . (f y \gg= g))$
- Destra
 - $\text{PutD } (x, y) \gg= (\lambda x \rightarrow f x \gg= g) =$
 $\text{PutD } (x, y) \gg= (f x \gg= g) =$
 $\text{PutD } ((\text{fst} . \text{getD} . (\lambda z \rightarrow (f z \gg= g))) x, (\text{snd} . \text{getD} . (\lambda z \rightarrow (f z \gg= g))) y) =$
 $\text{PutD } (\text{fst} . \text{getD} . (f x \gg= g), \text{snd} . \text{getD} . (f y \gg= g))$

Ex13:

Let us assume that we have defined the type:

`newtype Const c a = Const { getConst :: c }`

and that we already proved that it is an applicative functor.

1. Define the function ($\gg=$) for the functor Const c.
2. Prove the associativity law for ($\gg=$).

Soluzione:

1. Definizione di bind per il functor Const c:

si ha che c deve essere un Monade affinché possa provare che Const c è un Monade

$(\gg=) :: \text{Monoid } c \Rightarrow \text{Const } c \ a \rightarrow (a \rightarrow \text{Const } c \ b) \rightarrow \text{Const } c \ b$

si dovrebbe estrarre un valore di tipo a, ma Const c a contiene soltanto c (il valore a non è presente nella struttura), quindi ignora la funzione $f :: a \rightarrow \text{Const } c \ b$

- $\text{Const } x \gg= f$
= $\text{Const } x$
 $(\gg=) \ x \ f = \text{Const} . \text{getConst } x$

2. La legge dell'associatività per $\gg=$ afferma che $(m \gg= f) \gg= g = m \gg= (\lambda x \rightarrow f(x)) \gg= g$

- con il dato m (Const x) si ha:
- Sinistra
 - $(\text{Const } x \gg= f) \gg= g =$
 $(\text{Const } x \gg= g) =$
 $\text{Const } x$
- Destra
 - $\text{Const } x \gg= (\lambda x \rightarrow f(x)) \gg= g =$
 $\text{Const } x$

Ex14:

Let us assume that we have defined the type:

`newtype Identity a = Identity { runIdentity :: a }`

and that we already proved that it is an applicative functor.

1. Define the function $(\gg=)$ for the functor Identity a.
2. Prove the associativity law for $(\gg=)$.

Soluzione:

1. Definizione di bind per il functor Identity a:

$(\gg=) :: \text{Identity } a \rightarrow (a \rightarrow \text{Identity } b) \rightarrow \text{Identity } b$

- $\text{Identity } x \gg= f$
= $f \ x$
 $(\gg=) \ x \ f = f . \text{runIdentity } x$

2. La legge dell'associatività per $\gg=$ afferma che $(m \gg= f) \gg= g = m \gg= (\lambda z \rightarrow f(z)) \gg= g$

- con il dato m (Identity x) si ha:
- Sinistra
 - $(\text{Identity } x \gg= f) \gg= g =$
 $f \ x \gg= g$
- Destra

- Identity $x \gg= (\lambda z \rightarrow f(z)) \gg= g =$
 $(\lambda z \rightarrow f(z)) \gg= g) x =$
 $f(x) \gg= g$

Ex15:

Let us assume that we have defined the type:

`newtype Reader r a where`

`Reader :: (r -> a) -> Reader r a`

and that we already proved that it is an applicative functor.

1. Define the function $(\gg=)$ for the functor `Reader r`.
2. Write the essential description that justifies the given definition of $(\gg=)$.

Soluzione:

1. Definizione di `bind` per il functor `Reader r`:

$(\gg=) :: \text{Reader } r \ a \rightarrow (a \rightarrow \text{Reader } r \ b) \rightarrow \text{Reader } r \ b$

- `Reader g \gg= f`

`= Reader (\r ->`

`let a = g r -- Applica la funzione g all'ambiente r per ottenere il valore a`

`Reader h = f a -- Passa il valore a alla funzione f per ottenere un nuovo Reader h`

`in h r) -- Applica la funzione del nuovo Reader allo stesso ambiente r`

2. Tutte le computazioni condividono lo stesso ambiente in comune

Si esegue la prima funzione `g` usando l'ambiente `r`, si ottiene un valore `a`, che passandolo alla funzione `f` che produce una nuova computazione, e la si esegue nello stesso ambiente.

Entrambe le computazioni (originale `(Reader g)` e generata da `f (Reader h)`) vengono eseguite sullo stesso ambiente `r`, permettendo di concatenare operazioni che leggono dalla stessa fonte.

Ex16:

Let us assume that we have defined the type:

`newtype State s a = State { runState :: s -> (a, s) }`

1. Prove that it is a functor by defining the function `fmap`.
2. Write the essential description that justifies the given definition of `fmap`.

Soluzione:

Questo esercizio prova il Functor (dello `State s`)

`fmap :: (a -> b) -> F a -> F b`

1. `fmap :: (a -> b) -> State s a -> State s b`

- `fmap f (State g) = State (\s ->
let (val, nextState) = runState g s
in (f val, nextState))`

2. La funzione modifica il valore prodotto ma lascia invariato lo stato

Si accede alla funzione interna (dello State) `g` (di tipo `s -> (a, s)`) che rappresenta lo stato di computazione

Si crea una nuova funzione che accetta uno stato iniziale `s` (`\s -> ..`), che quando eseguita in `g` produce il valore attuale dello stato `val` e lo stato aggiornato `nextState`

La funzione `f` viene applicata al valore `val`, producendo un nuovo valore di tipo `b`

Lo stato risultato `nextState` si mantiene invariato, garantendo che `fmap` modifichi solo l'output della computazione

Quindi prima si occupa uno stato andando a prendere il suo stato interno e restituendo il valore e il secondo stato non va a toccare lo stato interno ma va a modificare il `val`

Ex17:

Let us assume that we have defined the type:

```
newtype State s a = State { runState :: s -> (a, s) }
```

and that we already proved that it is a functor.

1. Prove that it is applicative by defining the functions `pure` and `(< * >)`.
2. Write the essential description that justifies the given definition of `(< * >)`.

Soluzione:

Questo esercizio prova l'Applicative Functor

```
pure :: a -> F a
```

```
(< * >) :: F (a -> b)
```

```
-> F a
```

```
-> F b
```

1. `pure :: a -> State s a`

- `pure x = State (\s -> (x, s))`

2. `(< * >) :: State s (a -> b)`

```
-> State s a
```

```
-> State s b
```

- `State f < * > State x = State (\s ->
let (g, nextState) = runState f s
(y, nextState') = runState x nextState
in (g y, nextState'))`

3. Per la definizione di $< * >$ sullo stato si basa sul principio del passaggio sequenziale dello stato (uno stato alla volta)

Si esegue la prima computazione che produce la funzione g e cattura il nuovo stato risultante $nextState$

Si esegue la seconda computazione (passando il $nextState$ come input) generando l'argomento y e il nuovo stato $nextState'$

Infine si applica la funzione ottenuta g con il valore ottenuto y e si restituisce lo stato finale $nextState'$

Quindi si occupa prima dello State f con lo stato iniziale s e si ottiene una funzione g e il nuovo stato $nextState$, successivamente $nextState$ viene passato allo State x che produce un altro nuovo stato $nextState'$ e l'argomento y ; infine viene applicato alla funzione g il valore y e restituisce lo stato risultante $nextState'$

Ex18:

Let us assume that we have defined the type:

```
newtype State s a = State { runState :: s -> (a, s) }
```

and that we already proved that it is an applicative functor.

1. Define the function $(>>=)$ for the functor `State s`.
2. Write the essential description that justifies the given definition of $(>>=)$.

Soluzione:

Questo esercizio prova il Monade

```
(>>=) :: M a
```

```
-> (a -> M b)
```

```
-> M b
```

1. $(>>=) :: State s a$
-> $(a -> State s b)$
-> $State s b$
 - $State f >>= g = State (\s ->$
let $(a, nextState) = runState f s$
 $(State h) = runState g a$
in $h nextState)$

2. Lo stato prodotto dalla prima computazione diventa lo stato iniziale della seconda.

Si esegue la prima computazione dello State f passando lo stato iniziale s e generando un nuovo valore a e un nuovo stato $nextState$

Il valore a viene passato alla funzione g che restituisce una nuova computazione dello State

h.

Si esegue la nuova (seconda) computazione h passando il nuovo stato nextState